



US 20250286733A1

(19) **United States**

(12) **Patent Application Publication**
VILLEGAS et al.

(10) **Pub. No.: US 2025/0286733 A1**

(43) **Pub. Date: Sep. 11, 2025**

(54) **METHOD FOR ATTESTING AUTHENTICITY
OF COMPUTING DEVICE**

Publication Classification

(71) Applicant: **Nagravision Sàrl**,
Cheseaux-sur-Lausanne (CH)

(51) **Int. Cl.**
H04L 9/32 (2006.01)
H04L 9/06 (2006.01)
H04L 9/08 (2006.01)
(52) **U.S. Cl.**
CPC **H04L 9/3265** (2013.01); **H04L 9/0643**
(2013.01); **H04L 9/0891** (2013.01)

(72) Inventors: **Karine VILLEGAS**,
Cheseaux-sur-Lausanne (CH); **Yann
BIEBER**, Cheseaux-sur-Lausanne (CH)

(73) Assignee: **Nagravision Sàrl**,
Cheseaux-sur-Lausanne (CH)

(57) **ABSTRACT**

A method includes steps performed by a first computing device provisioned with a unique device secret, UDS, of generating an N chained certificate(s) attesting the authenticity of an N set(s) of device-specific data of the first computing device, and transmitting the N chained certificate(s) to a second computing device. The step of generating the N chained certificate(s) includes a generation process that includes: deriving, using a key ladder, a key from the UDS, generating an authentication tag from a set of device-specific data, using the key, generating a certificate that includes the set of device-specific data and the authentication tag, encrypting the set of device-specific data with the key, and providing the encrypted set of device-specific data as an input to the key ladder for a subsequent generation process.

(21) Appl. No.: **19/070,754**

(22) Filed: **Mar. 5, 2025**

(30) **Foreign Application Priority Data**

Mar. 5, 2024 (EP) 24161561

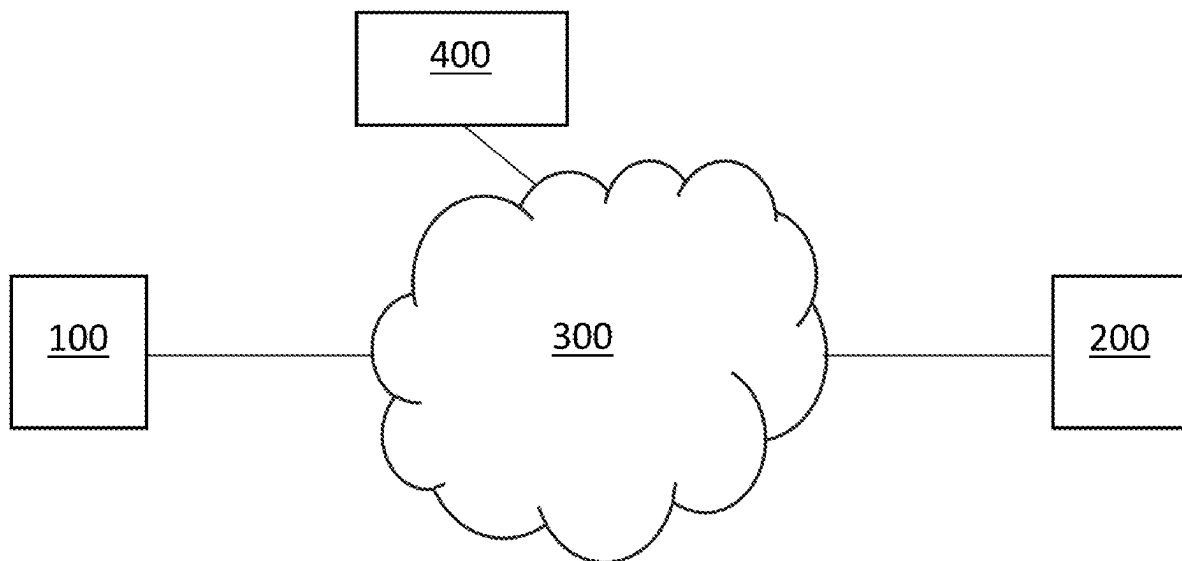


Figure 1

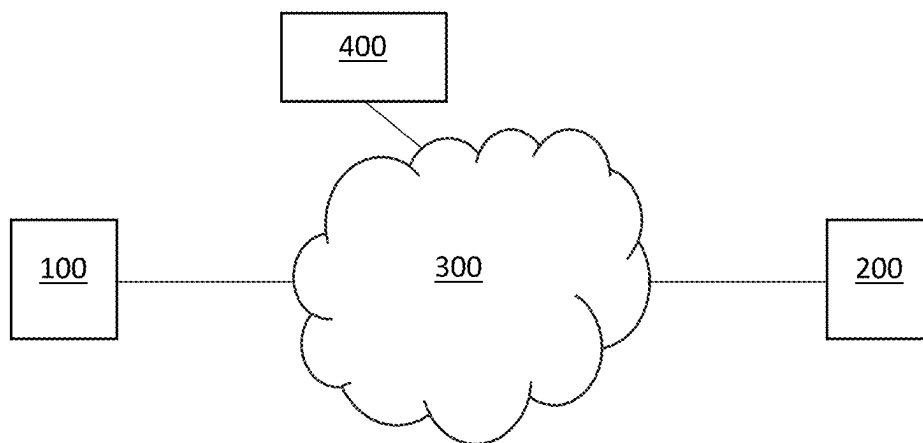


Figure 2

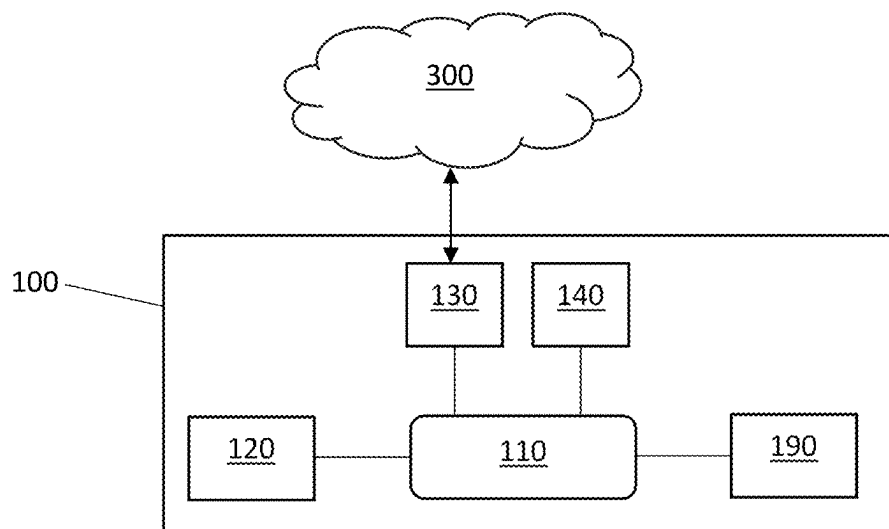


Figure 3

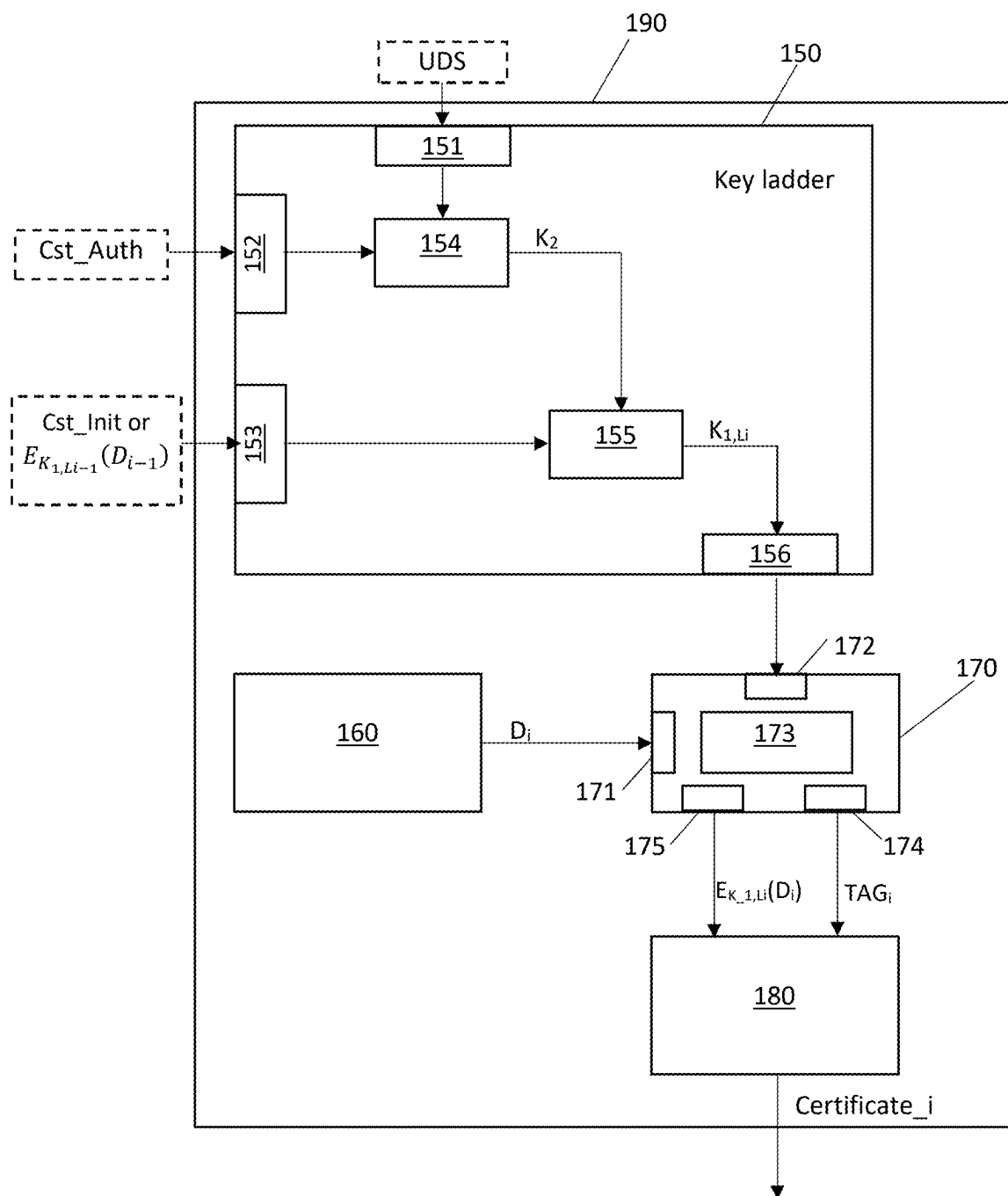


Figure 6

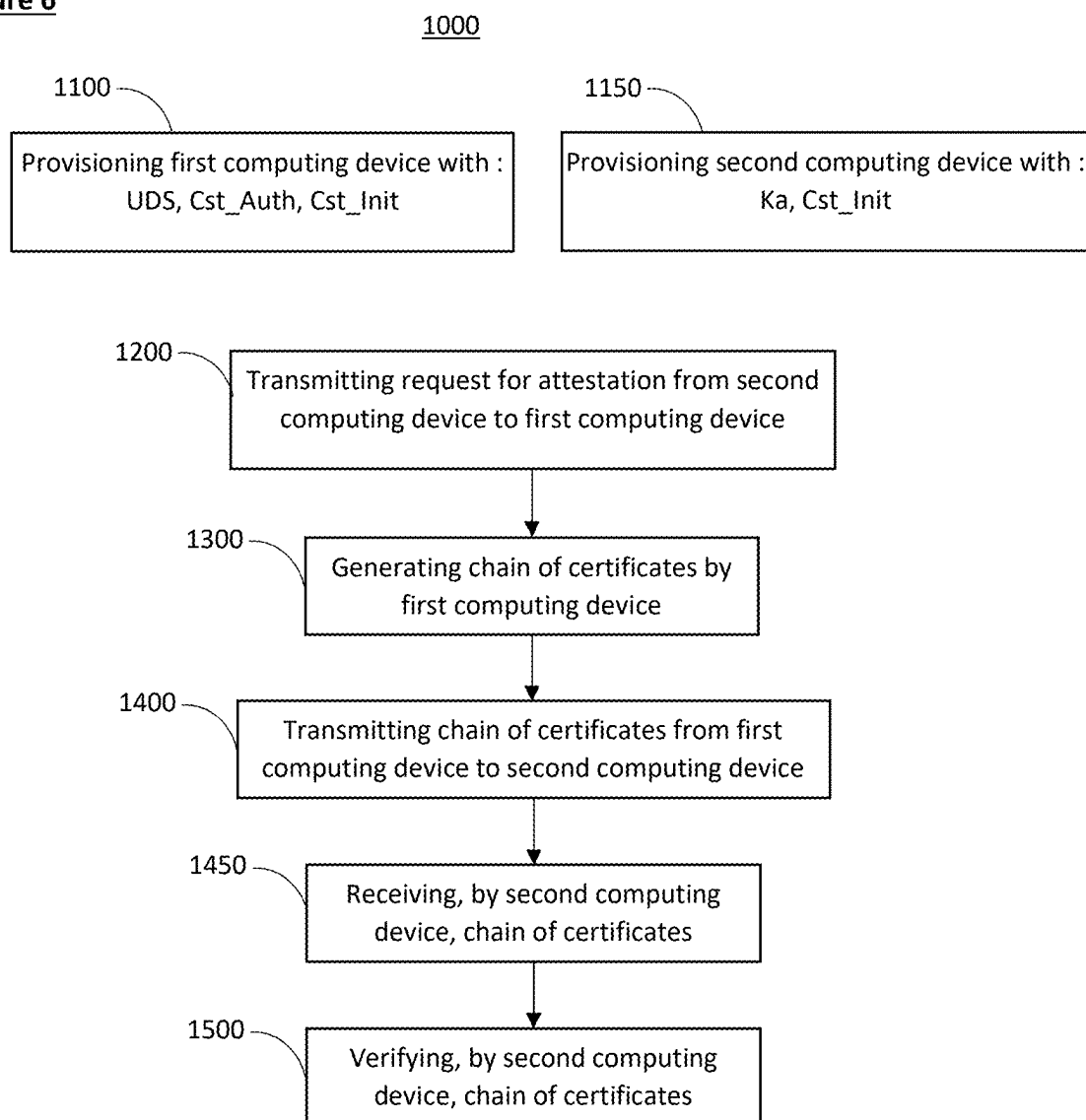


Figure 7A

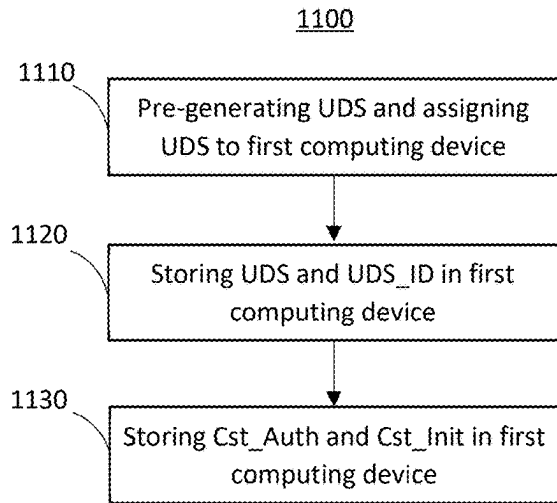


Figure 7B

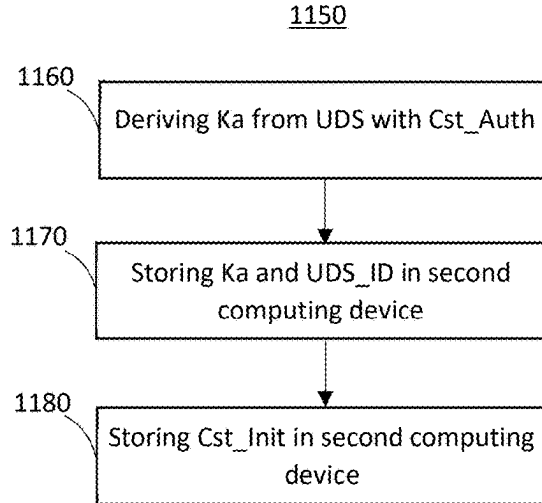


Figure 8

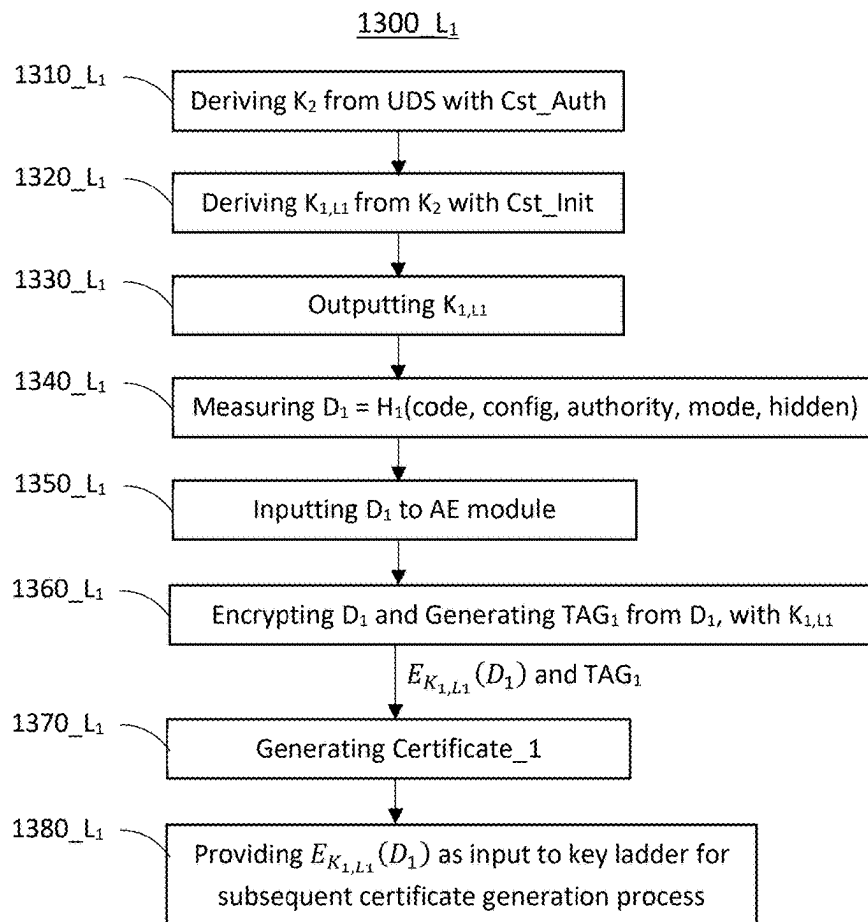


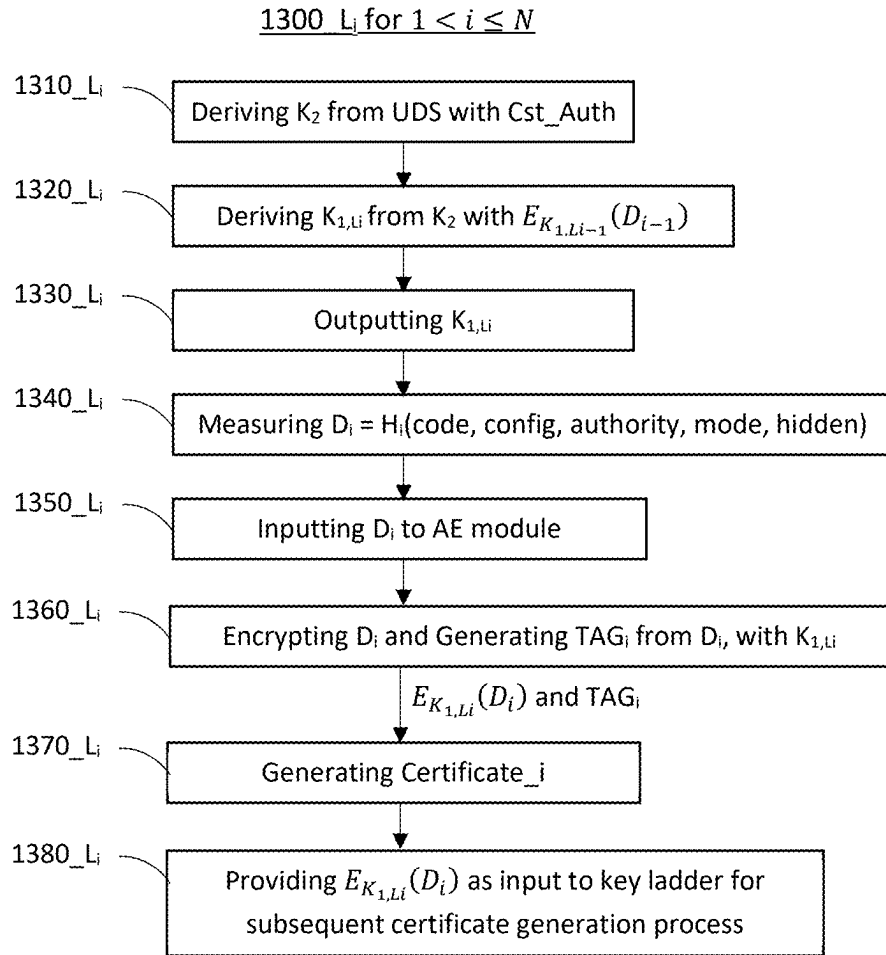
Figure 9

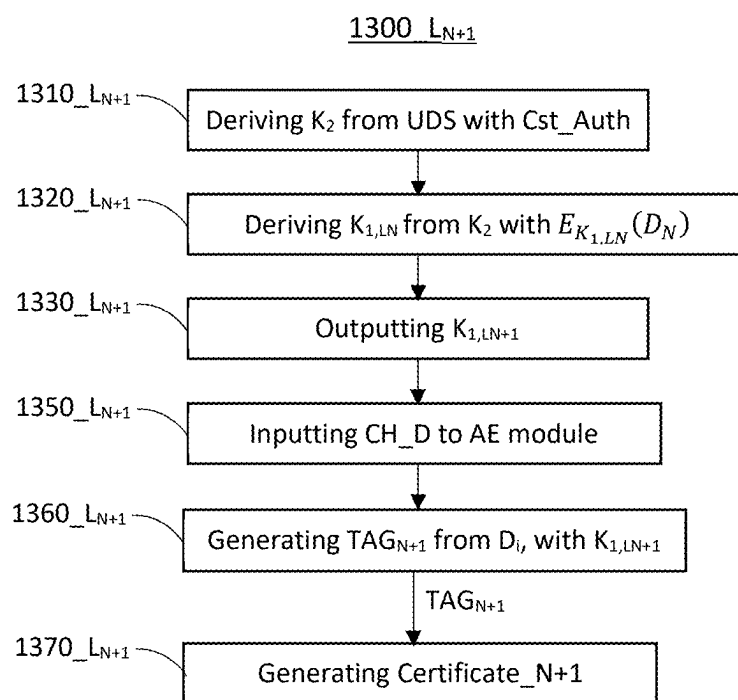
Figure 10

Figure 11

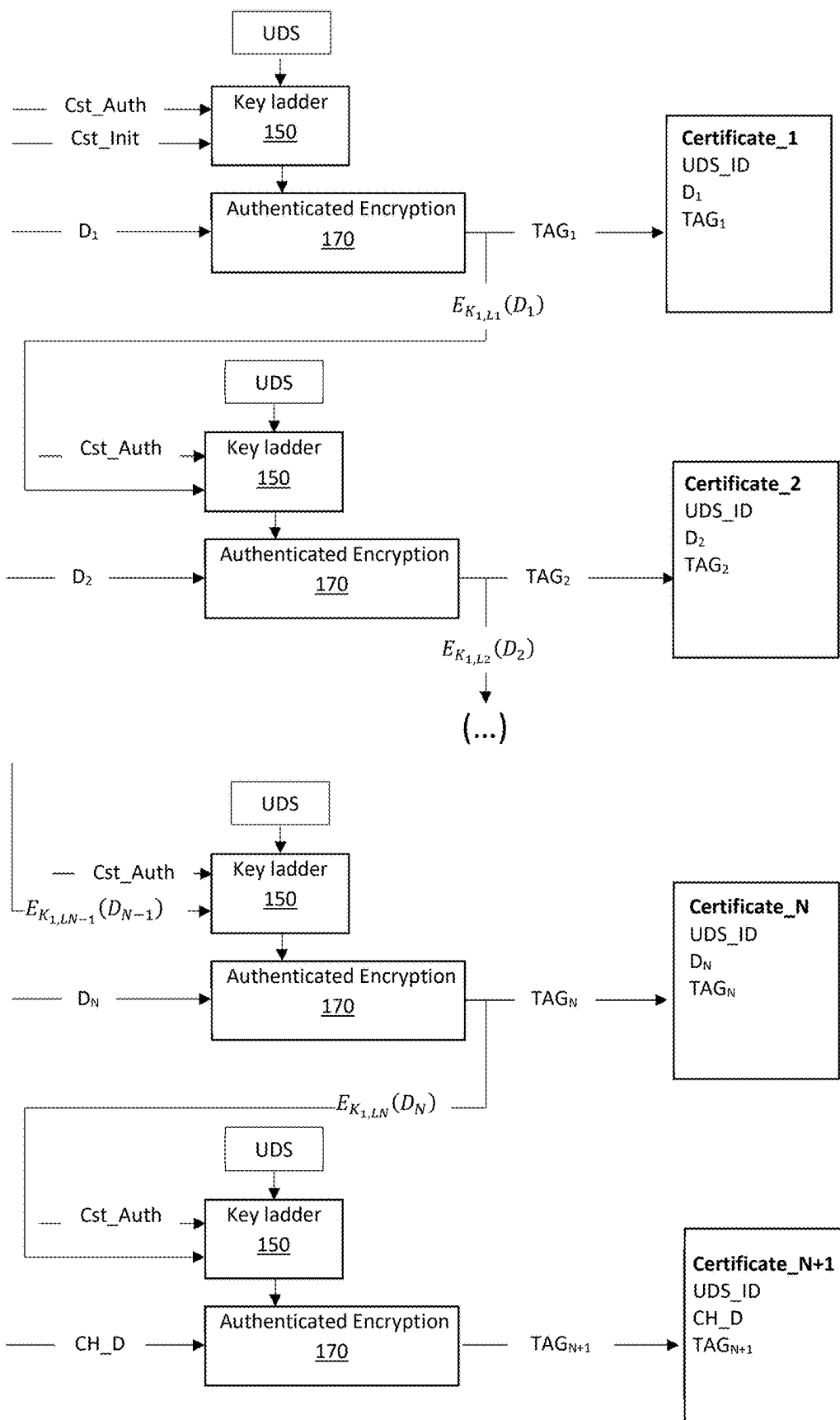


Figure 12

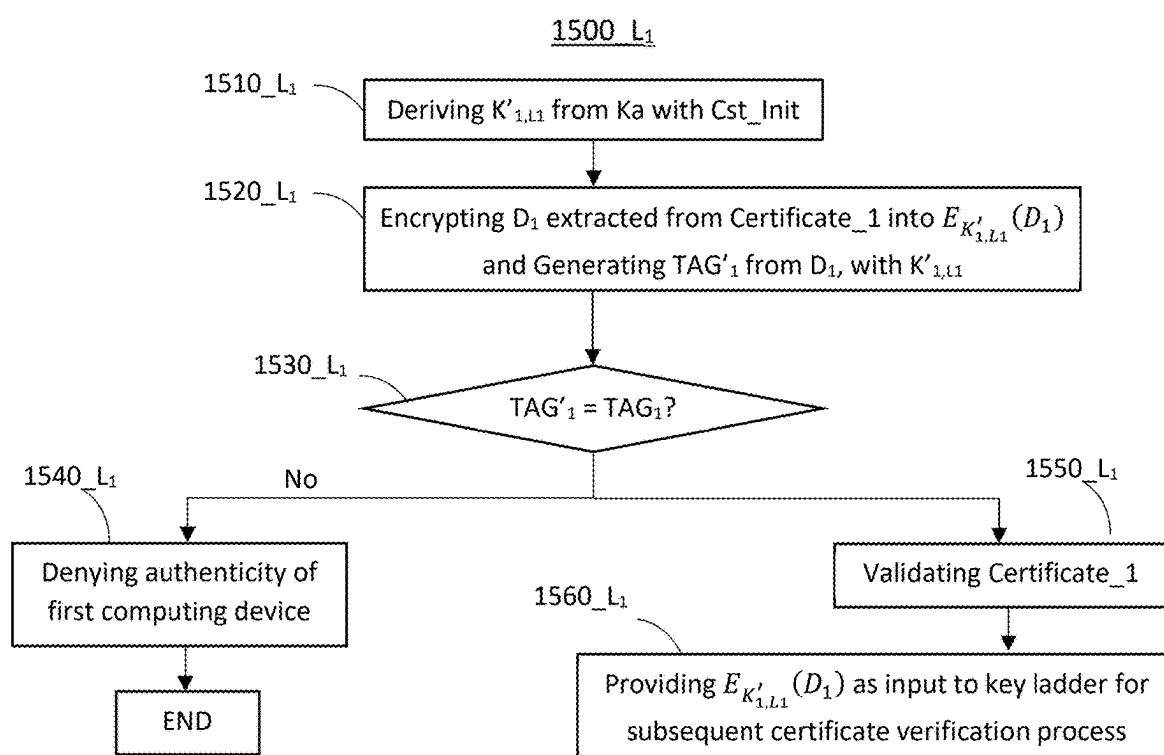


Figure 13

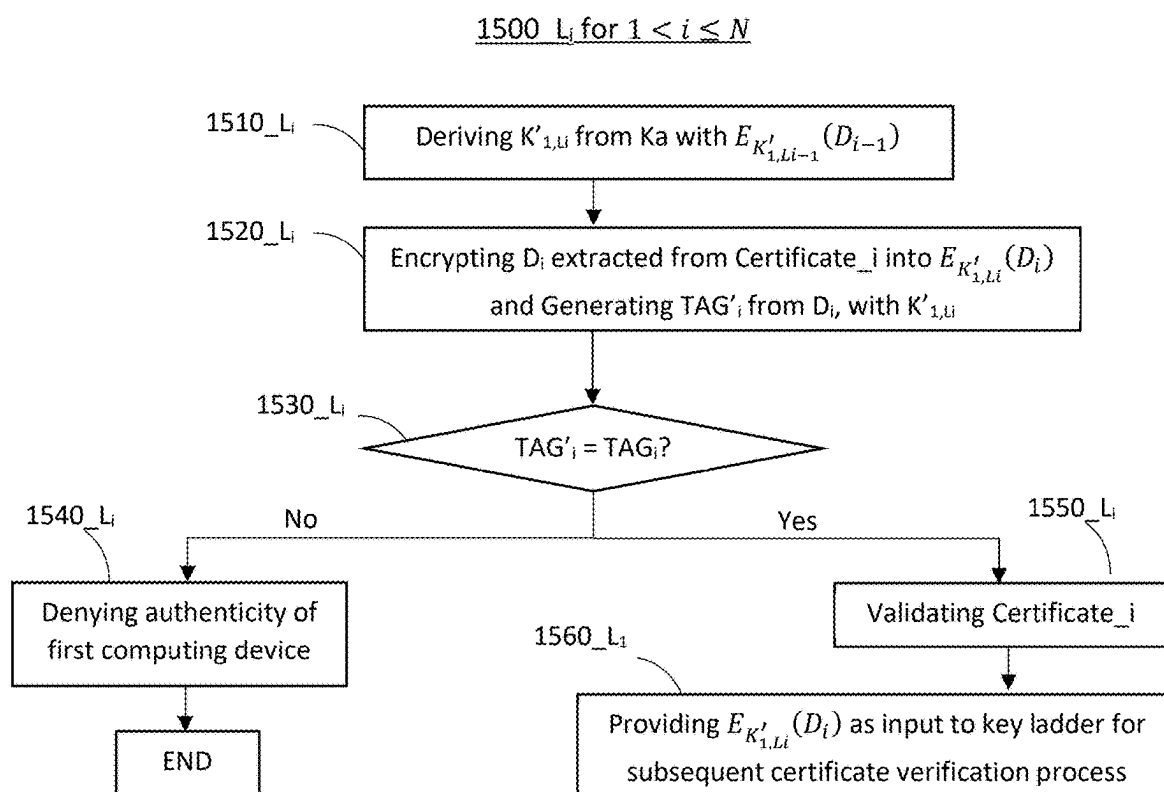


Figure 14

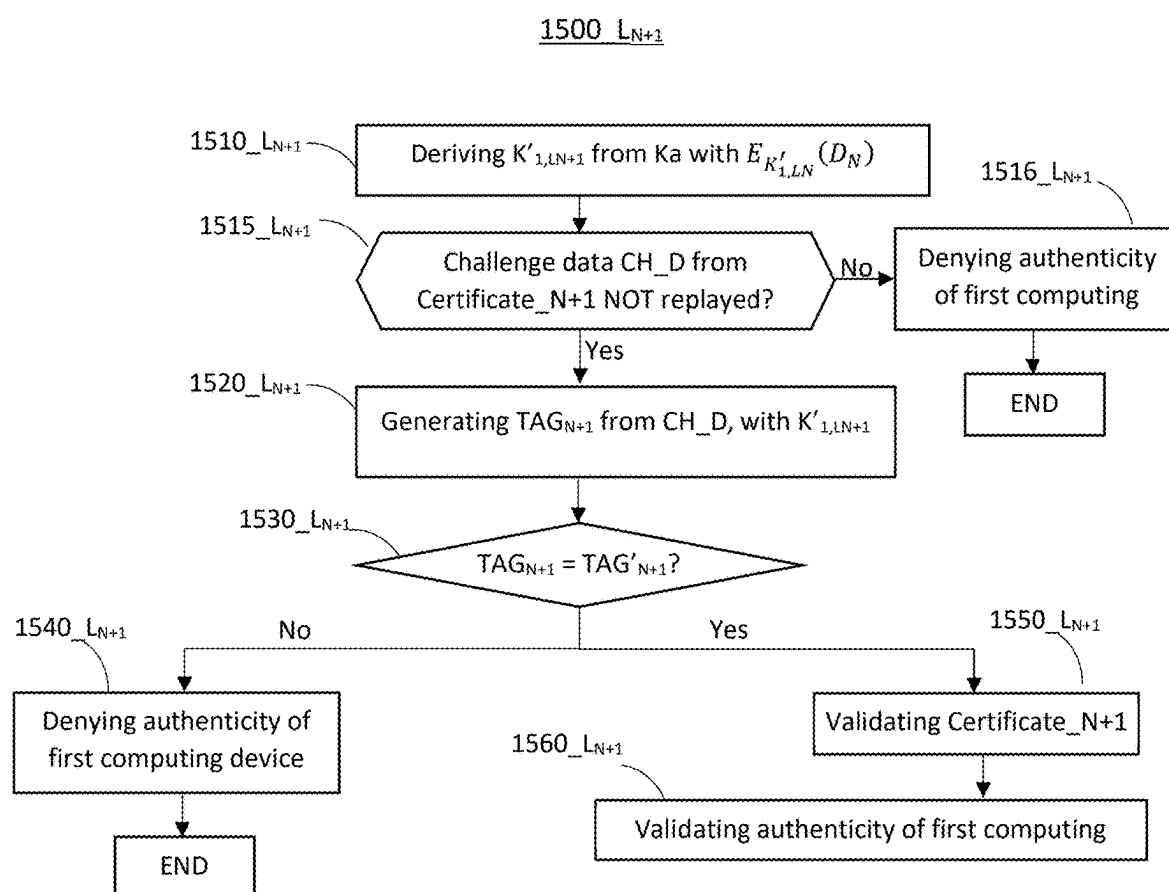


Figure 15

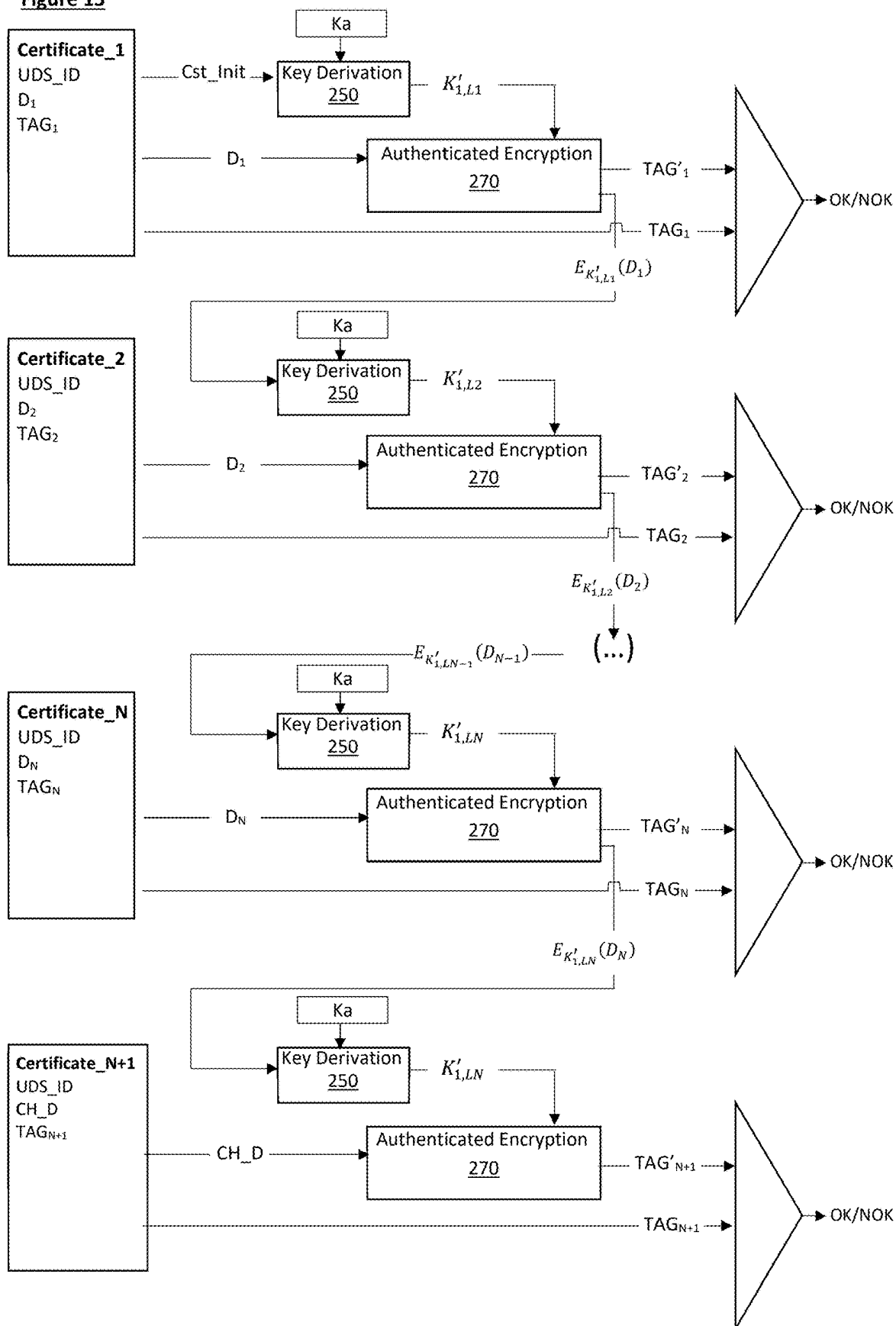
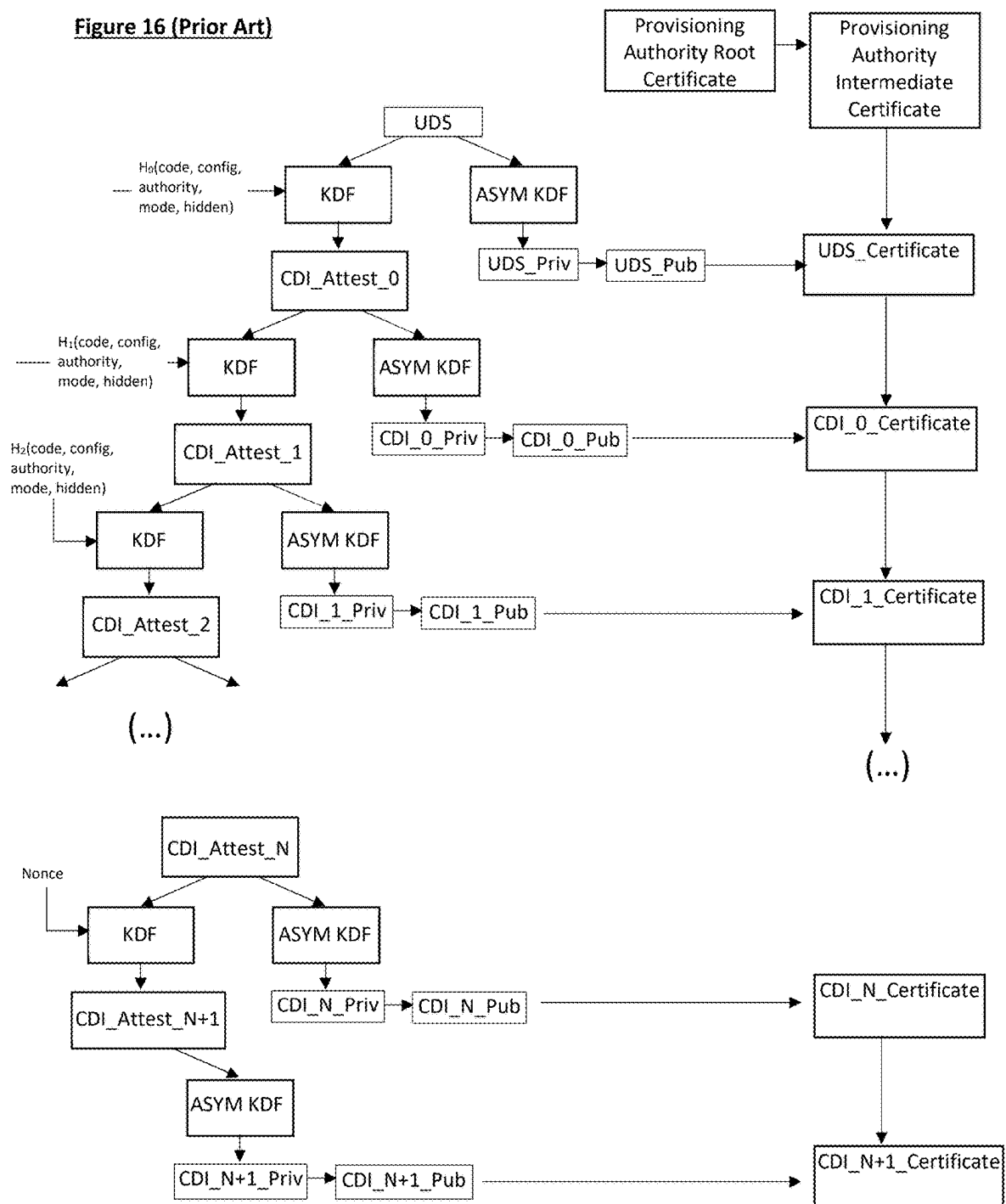


Figure 16 (Prior Art)



METHOD FOR ATTESTING AUTHENTICITY OF COMPUTING DEVICE

TECHNICAL FIELD

[0001] The present disclosure relates to device attestation for attesting authenticity of a computing device.

BACKGROUND

[0002] In the field of computer security, attestation or remote attestation refers to a method by which a first computing device (e.g., a client) authenticates its own software and/or hardware to a second computing device (e.g., a server). It allows the second computing device to obtain a level of trust in authenticity of the first computing device. This is crucial for establishing trust for example in IoT environments, where devices may be distributed and operated in untrusted environments.

[0003] A known method for performing remote attestation is based on Device Identifier Composition Engine (DICE) standard (specification for the Open

[0004] Profile for DICE v2.5) developed by the Trusted Computing Group (TCG). Each DICE-enabled device is assigned a Unique Device Secret (UDS) during manufacturing. This UDS is typically embedded in the device's hardware and is not accessible or modifiable by mutable software. Each DICE-enabled device is configured to execute a Device Identifier Composition Engine (DICE) that is a process which mixes the UDS with software hashes and other inputs to produce a Compound Device Identifier (CDI). The CDI is a value that represents a hardware and software composition measured by the DICE.

[0005] The DICE can be used for attestation to allow the DICE-enabled device to provide verifiable evidence of its identity and operating state, including hardware identity, software image, security-relevant configuration, operating environment, etc. For attestation, the DICE receives input values, each represented by a fixed length, that include: code (64 bytes), configuration data (64 bytes), authority data (64 bytes), mode decision (1 byte), hidden inputs (64 bytes). The DICE can derive an attestation CDI from the combination of all input values using a key derivation function (KDF). This CDI will change across software updates or configuration changes, which is appropriate for attestation.

[0006] A DICE-enabled device can generate a chain of certificates (i.e., a plurality of certificates that chain together), based on a layered DICE Open profile scheme that is illustrated in FIG. 16, to attest authenticity of the device.

[0007] At the top of FIG. 16, the DICE Open profile scheme includes the steps of:

[0008] deriving an asymmetric key pair UDS_Priv, UDS_Pub from the UDS used as a key with an asymmetric KDF;

[0009] generating an UDS certificate for the UDS-derived public key UDS_Pub, signed by an external authority;

[0010] measuring DICE input values $H_0(\text{code, config, authority, mode, hidden})$;

[0011] deriving, through a KDF, an attestation CDI CDI_Attest_0 from the UDS used as a key with the input values $H_0(\text{code, config, authority, mode, hidden})$;

[0012] deriving an asymmetric key pair CDI_0_Priv, CDI_0_Pub from the attestation CDI CDI_Attest_0 used as a key with an asymmetric KDF;

[0013] generating a CDI certificate CDI_0_Certificate for the CDI-derived public key CDI_0_Pub, that is signed using the UDS-derived private key UDS_Priv.

[0014] Then, at each lower level i with index i from 1 to N in FIG. 16, the DICE Open profile scheme includes the steps of:

[0015] measuring DICE input values $H_i(\text{code, config, authority, mode, hidden})$;

[0016] deriving, through the KDF, an attestation CDI CDI_Attest_ i from the previous attestation CDI CDI_Attest_ $i-1$ used as a key with the input values $H_i(\text{code, config, authority, mode, hidden})$;

[0017] deriving an asymmetric key pair CDI_ i _Priv, CDI_ i _Pub from the previous attestation CDI CDI_Attest_ $i-1$ used as a key with the asymmetric KDF;

[0018] generating a CDI certificate CDI_ i _Certificate for the CDI-derived public key CDI_ i _Pub, that is signed using the previous CDI-derived private key CDI_ $i-1$ _Priv.

[0019] The different layers of the DICE Open profile scheme can use different sets of input values $H_i(\text{code, config, authority, mode, hidden})$.

[0020] When generating certificates from level 1 to N , at each level i , the authority is the previous CDI key pair CDI_ $i-1$ _Priv, CDI_ $i-1$ _Pub.

[0021] The implementation of the DICE Open profile on devices has several drawbacks.

[0022] A main drawback is that the DICE Open profile scheme is vulnerable in case of key leakage and/or if a key has been compromised. The second computing device (e.g., a server) receives the chain of certificates from the first computing device (e.g., a client), and checks that the public key at one level $i-1$ validates the signature of the certificate at the next level i . In other words, the second computing device checks that the public key CDI_ $i-1$ _Pub validates the signature of CDI_ i _certificate that has been signed with the private key CDI_ $i-1$ _Priv. But the second computing device cannot check if this private key CDI_ $i-1$ _Priv has been correctly derived from the secret CDI_Attest_ $i-1$. If, at one layer of the DICE Open profile scheme, the symmetric secret (i.e., the attestation CDI) has been compromised, the second computing cannot detect that the CDI-derived private key has been generated from a compromised secret (i.e., a compromised attestation CDI) with the asymmetric KDF.

[0023] Thus, if the attestation CDI derived from all the input values is compromised by a malicious person at any level i of the DICE Open profile scheme, this attestation CDI can however be used as a secret key to derive a CDI-derived key pair with the asymmetric KDF, and this key pair can be used to sign the next CDI certificate. The second computing device will validate that this next certificate CDI certificate is signed by the previous CDI-derived private key using the corresponding CDI-derived public key and thus chained to the previous CDI certificate, but cannot detect that the attestation CDI, from which the CDI-derived key pair was derived, has been compromised. In such a case, the second computing device will incorrectly validate the chain of certificates and attest the authenticity of the first computing device, which is a problem.

[0024] The DICE Open profile has other drawbacks:

[0025] It requires specific hardware and software means supporting DICE.

[0026] It requires to report all the UDS_certificates. The UDS and the associated certificate UDS_certificate are pre-

generated and injected during a manufacturing process in each DICE-enabled device, in a secure environment. The UDS certificates of all manufactured devices must be reported to an entity that will make them available to the second computing device (e.g., a server) responsible for validating the certificate chain.

[0027] Therefore, there is a need for improving the situation. In particular, it is desired for the attestation scheme to be more robust to key compromise and/or key leakage.

SUMMARY

[0028] The scope of protection is set out by the independent claims. The embodiments, examples and features, if any, described in this specification that do not fall under the scope of the protection are to be interpreted as examples useful for understanding the various embodiments or examples that fall under the scope of protection.

[0029] The present disclosure concerns a method for attesting authenticity of a first computing device, said first computing device being provisioned with a unique device secret, UDS, the method including the steps, performed by the first computing device, of:

- [0030] generating an N chained certificate[s], the N chained certificate[s] attesting the authenticity of an N set[s] of device-specific data of the first computing device; transmitting the N chained certificate[s] to a second computing device;
- [0031] wherein the step of generating the N chained certificate[s] comprises a generation process that includes the steps of:
- [0032] A. deriving, using a key ladder, a key from the UDS;
- [0033] B. generating an authentication tag from a set of device-specific data, using the key;
- [0034] C. generating a certificate, said certificate including the set of device-specific data and the authentication tag;
- [0035] D. encrypting the set of device-specific data with the key; and
- [0036] E. providing the encrypted set of device-specific data as an input to the key ladder for a subsequent generation process.
- [0037] The method may include the steps of
- [0038] successively executing N generation processes of generating one of N chained certificates, each certificate attesting the authenticity of one of N sets of device-specific data of the first computing device;
- [0039] transmitting the N chained certificates to a second computing device;
- [0040] wherein each generation process includes the steps of:
- [0041] A. deriving, using a key ladder, at least one key from the UDS;
- [0042] B. generating an authentication tag from one of the N sets of device-specific data, using the at least one derived key; and
- [0043] C. generating the certificate, said certificate including said one of the N sets of device-specific data and the authentication tag; and from the first to N-1th certificate generation process:
- [0044] D. encrypting the one of the N sets of device-specific data with the at least one derived symmetric key; and

[0045] E. providing the encrypted set of device-specific data as an input to the key ladder for the subsequent generation process.

[0046] The present method for attesting authenticity of a computing device relies on a key ladder. Such key ladder can be found in many computing devices, for example in chip-sets. The generation of the chained certificates is layered: it includes successively executing generation processes of generating one of a plurality of chained certificates. At each layer, the key ladder derives a key, for example a symmetric key, from the UDS used as a key and the first computing device generates a certificate including an authentication tag (e.g., a CMAC (block cipher-based message authentication code)), generated with said UDS-derived key. Furthermore, at each layer, a set of device-specific data is encrypted with the UDS-derived key produced by the key ladder, and then, at the subsequent layer, this encrypted set of device-specific data is used as input value by the key ladder to derive another key from the UDS. This allows to chain the certificates that each include an authentication tag, generated with the UDS-derived key from the key ladder.

[0047] The present method allows the second device to validate the keys used to create the certificate chain. Furthermore, if for example one encrypted set of device-specific data produced during one certificate generation process and used as input to the key ladder for the subsequent certificate generation process, or an UDS-derived key, or even the UDS is compromised, the rest of the certificate chain generation cannot be simply implemented in software without breaking the internals of the key ladder and the UDS key. It would not be sufficient for an attacker to compromise the encrypted data produced in a certificate generation process or an UDS-derived key or even the UDS. The attacker would also need to compromise the key ladder implementation details to produce a chain of certificates allowing the second computing device to carry out a valid attestation. The security of the present attestation scheme cannot be simply broken if the UDS key or an UDS-derived key or the encrypted data produced during one certificate generation process and used as input to the key ladder for the subsequent certificate generation process is leaked or compromised. As a result, the present attestation method is extremely robust against attacks.

[0048] The present method can be easily implemented on existing devices that already include a key ladder.

[0049] According to an embodiment, in the step A., the key ladder has at least two levels and carries out the steps of:

- [0050] receiving a higher-level input value; and
- [0051] deriving a higher-level key from the UDS used as a key or from a UDS-derived key with the higher-level input value.

[0052] Thanks to that, the UDS does not need to be provided to the second computing device. The second computing device only needs to have the higher-level key derived from the UDS, that is less sensitive data and can be renewed in case of exposure.

[0053] Advantageously, the higher-level input value received by the key ladder includes a predetermined constant that is the same for all the generation processes.

[0054] In an embodiment, in the step A., the key ladder further carries out the step of deriving the key from the higher-level key with a lower-level input value.

[0055] In an embodiment, in the step A.,

[0056] for a first generation process, the key ladder receives a predetermined initial value as lower-level input value;

[0057] for each of subsequent generation processes, the key ladder receives the encrypted set of device-specific data provided by the preceding generation process as lower-level input value.

[0058] In a first embodiment, the steps B. and D. are combined by executing an authenticated encryption algorithm so as to simultaneously encrypt the set of device-specific data and generate the authentication tag from the set of device-specific data with the key derived from the UDS in the step A.

[0059] In a second embodiment, in the step A., the key ladder derives two keys, different from one another, from the UDS used as a key, and:

[0060] in the step B., the authentication tag is generated with one of the two keys;

[0061] in the step C., the set of device-specific data is encrypted with the other of the two keys.

[0062] Advantageously, the two keys can be separately derived from the UDS by the key ladder using respectively a first higher-level input value and a second higher-level input value, different from one another.

[0063] The authentication tag can be generated with a block cipher-based message authentication code algorithm.

[0064] In an embodiment, the method may comprise a generation process of generating a final certificate, said generation process including the steps of:

[0065] receiving challenge data from the second computing device;

[0066] receiving, by a key ladder, an encrypted set of device-specific data provided by the preceding generation process;

[0067] deriving, by the key ladder, a key from the UDS using said encrypted set of device-specific data as input data;

[0068] generating a final tag from the challenge data using the key;

[0069] generating the final certificate including the challenge data and the final tag.

[0070] For example, the step A. can be performed by executing a cryptographic algorithm including at least one of a decryption algorithm and an encryption algorithm, at each of a plurality of levels of the key ladder.

[0071] The present disclosure also concerns a method for attesting the authenticity of a first computing device provisioned with a unique device secret UDS, by a second computing device provisioned with a key that is derived from the UDS of the first computing device, the method comprising the steps of:

[0072] receiving, by the second computing device, from the first computing device, an N chained certificate[s] generated by performing the method previously defined;

[0073] verifying, by the second computing device, the validity of the N chained certificate[s];

[0074] wherein the step of verifying the validity of the N chained certificate[s] comprises a verification process, performed by the second computing device, that includes the steps of:

[0075] F. deriving, by a key derivation module, a key from said provisioned key;

[0076] G. generating a verification tag from a set of device-specific data included in a certificate to be verified using the key;

[0077] H. comparing the verification tag and an authentication tag included in the certificate to be verified;

[0078] I. if the verification tag and the authentication tag match, validating the certificate;

[0079] J. encrypting the set of device-specific data included in the certificate to be verified with the key; and

[0080] K. providing the encrypted set of device-specific data as an input to the key derivation module for a subsequent verification process.

[0081] In an embodiment, the method can include the steps of:

[0082] receiving by the second computing device, from the first computing device, N chained certificates generated by performing the method previously defined;

[0083] verifying by the second computing device the validity of the N chained certificates;

[0084] wherein the step of verifying the validity of the N chained certificates includes successively executing N verification processes of verifying one certificate of the N chained certificates, and each verification process includes the steps, performed by the second computing device, of:

[0085] F. deriving, by a key derivation module, at least one symmetric key from said at least one provisioned symmetric key;

[0086] G. generating a verification tag from the set of device-specific data included in the certificate to be verified using the at least one derived symmetric key;

[0087] H. comparing the verification tag and the authentication tag included in the certificate to be verified;

[0088] I. if the verification tag and the authentication tag match, validating the certificate; and from the first to N-1th certificate verification process:

[0089] J. encrypting the set of device-specific data included in the certificate to be verified with the at least one derived symmetric key; and

[0090] K. providing the encrypted set of device-specific data as an input to the key derivation module for the subsequent certificate verification process.

[0091] In an embodiment, said provisioned key may correspond to the higher-level key generated by the key ladder in the step A of the method previously defined.

[0092] Advantageously, in a first verification process, the step F includes receiving as input, by the key derivation module, a predetermined initial value that is the same as the predetermined initial value received by the key ladder as lower-level input in the first generation process of the method as previously defined.

[0093] The present disclosure also concerns:

[0094] a first computing device comprising at least one key ladder and means adapted to execute the steps of the method previously defined;

[0095] a second computing device, comprising at least one key derivation module and means adapted to execute the steps of the method previously defined;

[0096] a system comprising the above-defined first computing device and the above-defined second computing device;

- [0097] a computer program comprising instructions to cause the above-defined first computing device to execute the steps of the method previously defined;
- [0098] a computer-readable medium having stored thereon the above-defined computer program;
- [0099] a computer program comprising instructions to cause the above-defined second computing device to execute the steps of the method previously defined; and
- [0100] a computer-readable medium having stored thereon the above-defined computer program.

BRIEF DESCRIPTION OF THE DRAWINGS

- [0101] Example embodiments will become more fully understood from the detailed description given herein below and the accompanying drawings, which are given by way of illustration only and thus are not limiting of this disclosure.
- [0102] FIG. 1 illustrates a system including a first computing device and a second computing device.
- [0103] FIG. 2 shows a block diagram of the first computing device, according to an embodiment.
- [0104] FIG. 3 shows a block diagram of an attestation engine of the first computing device, according to an embodiment.
- [0105] FIG. 4 shows a block diagram of the second computing device, according to an embodiment.
- [0106] FIG. 5 shows a block diagram of a verification engine of the second computing device, according to an embodiment.
- [0107] FIG. 6 shows a flow chart of an attestation method for attesting authenticity of a first computing device, according to an embodiment.
- [0108] FIGS. 7A and 7B represent flow charts of a provisioning process for the first computing device and a provisioning process for the second computing device, according to an embodiment.
- [0109] FIG. 8 represents a flow chart of a first or top certificate generation process, according to a first Embodiment.
- [0110] FIG. 9 represents a flow chart of a subsequent certificate generation process, according to the first Embodiment.
- [0111] FIG. 10 represents a flow chart of a last or bottom certificate generation process, according to the first Embodiment.
- [0112] FIG. 11 represents a block diagram of a method for generating a chain of certificates, carried out by the first computing device, according to the first Embodiment.
- [0113] FIG. 12 represents a flow chart of a first certificate verification process, according to the first Embodiment.
- [0114] FIG. 13 represents a flow chart of a subsequent certificate verification process, according to the first Embodiment.
- [0115] FIG. 14 represents a flow chart of a last or bottom certificate verification process, according to the first Embodiment.
- [0116] FIG. 15 represents a block diagram of a method for verifying a chain of certificates, carried out by the second computing device, according to the first Embodiment.
- [0117] FIG. 16 represents a block diagram of a method for generating a chain of certificates, carried out by a first computing device, according to the prior art.
- [0118] It should be noted that these drawings are intended to illustrate various aspects of devices, methods and structures used in example embodiments described herein. The

use of similar or identical reference numbers in the various drawings is intended to indicate the presence of a similar or identical element or feature.

DETAILED DESCRIPTION

[0119] The following detailed description describes various features and functions of the disclosed systems and methods with reference to the accompanying figures. The illustrative system, device and method embodiments described herein are not meant to be limiting. It may be readily understood by those skilled in the art that certain aspects of the disclosed systems, devices and methods can be arranged and combined in a wide variety of different configurations, all of which are contemplated herein.

First Embodiment

[0120] FIG. 1 shows a system including a first computing device **100** and a second computing device **200**, according to a first Embodiment. The two computing devices **100**, **200** can communicate with one another through a communication network and/or a communication link **300**.

[0121] The computing devices **100**, **200** may correspond to any physical or virtual computing device, such as a mobile device (e.g., a smartphone), a desktop computer, a laptop computer, a IoT device, a tablet, a game console, a server, etc. Each computing device may be a single entity, multiple entities or a distributed system (e.g., a cloud system). In an illustrative and non-limitative example, the first device and the second device may be a client device and a server device, respectively.

[0122] The communication network or communication link **300** can be of any type (e.g., mobile telecommunications network(s), wireless local area network (WLAN or Wi-Fi), worldwide interoperability for microwave access (WiMAX), Bluetooth®, personal communications services (PCS), ZigBee®, wideband code division multiple access (WCDMA), systems using ultra-wideband (UWB) technology, sensor networks, mobile ad-hoc networks (MANETs), Internet Protocol multimedia subsystems (IMS), . . . or any combination thereof).

[0123] FIG. 2 schematically illustrates the first computing device **100**, according to the first Embodiment. The first computing device **100** may include at least one processor **110**, at least one memory or memory means **120** connected to the processor **110**, and at least one communication interface **130** connected to the processor **110** and configured to communicate through a communication network and/or a communication link **300**. Optionally, the first computing device **100** may further include one or more user interfaces **140** (e.g., keyboard, mouse, display screen, etc.) connected to the processor **110**.

[0124] The memory or memory means **120** may be or include a random-access memory (RAM), cache memory, non-volatile memory, backup memory (e.g., programmable or flash memories), read-only memory (ROM), a hard disk drive (HDD), a solid state drive (SSD) or any combination thereof. The ROM of the memory means **120** may be configured to store, amongst other things, an operating system of the device **100** and/or one or more computer program code of one or more software applications. The RAM of the memory means **120** may be used by the processor **110** for the temporary storage of data.

[0125] The processor 110 may be configured to store, read, load, and/or execute instructions (e.g., program instructions, computer program code) stored in the memory means 120 such that, when the instructions are executed by the processor 110, it causes the computing device 100 to perform one or more or all steps of a method described herein for the concerned device 100.

[0126] In the present disclosure, the first computing device 100 includes an attestation engine 190. A role of this attestation engine 190 is to produce or generate a chain of digital certificates for attesting authenticity of the first computing device 100, i.e., for providing verifiable evidence of the authenticity of the first computing device 100 to the second computing device 200 or any other party.

[0127] FIG. 3 illustrate the attestation engine 190 according to the first Embodiment.

[0128] As shown in FIG. 3, the attestation engine 190 may include a key ladder 150 configured to derive multiple keys from one master key. This key ladder 150 may be implemented with hardware means and/or software means (i.e., hardware, or software, or a mix of software and hardware). The key ladder 150 may include at least two stages or levels to derive at least two keys from the master key. As an illustrative and non-limitative example, the key ladder 150 may be based on the specifications ETSI TS 103 162 V1.1.1 (2010-10).

[0129] FIG. 3 schematically represents a functional diagram of the key ladder 150 according to an embodiment that is only illustrative and is not limitative. In this embodiment, the key ladder 150 has two stages or levels. The key ladder 150 may include:

- [0130] a master key input interface 151,
- [0131] a higher-level input interface 152,
- [0132] a lower-level input interface 153,
- [0133] a higher-level key derivation module 154,
- [0134] a lower-level key derivation module 155, and
- [0135] an output interface 156.

[0136] The higher-level key derivation module 154 may be connected to the master key input interface 151 and to the higher-level input interface 152. The lower-level key derivation module 155 may be connected to an output of the higher-level key derivation module 154, to the lower-level input interface 153, and to the output interface 156.

[0137] The higher-level derivation module 154 is configured to derive an intermediate key, or higher-level derived key, from the key here received through the master key input interface 151, with a higher-level input value received through the input interface 152. The lower-level derivation interface 155 is configured to derive a final key, or lower-level derived key, from this intermediate key with a lower-level input value received through the input interface 153.

[0138] The key derivation modules 154, 155 may be cryptographic modules configured to perform cryptographic operations (i.e., execute one or more cryptographic algorithms) to derive a key from another key with an input value. For example, the key derivation modules 154, 155 may be configured to execute a decryption algorithm or an encryption algorithm such as AES (Advanced Encryption Standard), Triple DES (Data Encryption Security), In an illustrative and non-limitative embodiment, the key derivation modules 154, 155 are two decryption modules. The operation of the key ladder 150 will be described in more detail in the description of the method.

[0139] In other embodiments, the key ladder 150 may include more than two stages or levels. For example, the key ladder 150 may include at least one additional key derivation module, in addition the two key derivation modules 154, 155. The additional key derivation module(s) may be configured to derive an intermediate key from the master key and provide this intermediate key to the higher-level derivation module 154, instead of the master key, for derivation purpose.

[0140] More generally, in some embodiments, the key ladder 150 may be configured to execute one or more additional cryptographic or transformation algorithms, in addition to those carried out by the two key derivation modules 154, 155, at any stage of the key ladder 150, for example to transform the UDS, and/or at least one input value received by the key ladder 150, and/or at least one UDS-derived key produced by the key ladder 150. The additional cryptographic or transformation algorithm(s) may include at least one of an encryption algorithm, a decryption algorithm, a key derivation function, or any combination thereof. The additional cryptographic or transformation algorithms may use global cryptographic keys (i.e., keys that can be provisioned to multiple first computing devices). In any case, the second computing device 200 should be configured in a similar manner to the first computing device 100 to be able to perform similar computations.

[0141] The attestation engine 190 may include a measurement module or collection module 160 configured to measure or collect or determine a set of device-specific data D_i , as will be described in more detail in the description of the method.

[0142] In the first Embodiment, the attestation engine 190 may include an authenticated encryption (AE) module 170. This AE module 170 may comprise a data input interface 171 configured to receive input data to encrypt, a key input interface 172, an authenticated encryption block 173 connected to the data input interface 171 and to the key input interface 172, and two output interfaces 174, 175. The data input interface 171 may be connected to the measurement module 160. The key input interface 172 may be connected to the output interface 156 of the key ladder 150, and is configured to receive a secret key (i.e., a symmetric key).

[0143] The authenticated encryption (AE) block 173 may be configured to execute an authenticated encryption algorithm to encrypt the input data and generate an authentication tag from this input data, with the secret key received through the key input interface 172. As illustrative and non-limitative examples, the AE block 173 may use an authenticated encryption algorithm such as GCM (Galois/Counter Mode), CCM (Counter with CBC-MAC) or a sponge-based construction.

[0144] The authentication tag may be for example a message authentication code or MAC, for example a CMAC (i.e., a block cipher-based message authentication code), HMAC (i.e., Hash-based message authentication code), or KMAC (i.e., KECCAK Message Authentication Code). The first output interface 174 may be configured to output this authentication tag.

[0145] The second output interface 175 may be configured to output the encrypted data.

[0146] The attestation engine 190 may include a certificate generator 180 configured to create or generate each certificate Certificate_i of a chain of certificates, as will be described in more detail in the description of the method.

[0147] FIG. 4 schematically illustrates the second computing device 200, according to the first Embodiment. The second computing device 200 may include at least one processor 210, at least one memory or memory means 220, and at least one communication interface 230 connected to the processor 210 and configured to communicate through a communication network 300 or a communication link. Optionally, the second computing device 200 may further include one or more user interfaces 240 (e.g., keyboard, mouse, display screen, etc.) connected to the processor 210.

[0148] The memory or memory means 220 may be or include a random-access memory (RAM), cache memory, non-volatile memory, backup memory (e.g., programmable or flash memories), read-only memory (ROM), a hard disk drive (HDD), a solid-state drive (SSD) or any combination thereof. The ROM of the memory means 220 may be configured to store, amongst other things, an operating system of the device 200 and/or one or more computer program code of one or more software applications. The RAM of the memory 220 may be used by the processor 210 for the temporary storage of data.

[0149] The processor 210 may be configured to store, read, load, and/or execute instructions (e.g., program instructions, computer program code) stored in the memory means 220 such that, when the instructions are executed by the processor 210, it causes the computing device 200 to perform one or more or all steps of a method described herein for the concerned device 200.

[0150] In the present disclosure, the second computing device 200 includes a verification engine 290. A role of this verification engine 290 is to verify the validity of a chain of digital certificates associated with the first computing device 100, or with another party, for attesting the authenticity of this first computing device 100 or other party.

[0151] FIG. 5 illustrate the verification engine 290 according to the first Embodiment.

[0152] The verification engine 290 may include a key derivation module 250. This key derivation module 250 may include a key input interface 251, a value input interface 252 and an output interface 253. The key input interface 251 may be configured to receive a key Ka that corresponds (i.e., for example is equal to or the same as) to the higher-level UDS-derived key (i.e., intermediate key) provided at the output of the higher-level key derivation module 154 of the key ladder 150 of the first computing device 100. The key Ka may be pre-stored in the memory means 220. The key derivation module 250 may be similar to the lower-level key derivation module 155 of the key ladder 150 of the first computing device 100. It may be configured to execute the same cryptographic algorithm(s) for deriving a key from another key as this lower-level key derivation module 155, as will be described in more detail in the description of the method.

[0153] The verification engine 290 may include an authenticated encryption (AE) module 270. This module 270 may be similar to the AE encryption module 170 of the first computing device 100. It may comprise a data input interface 271 configured to receive input data to encrypt, a key input interface 272, an encryption block 273 connected to the data input interface 271 and the key input interface 272, and two output interfaces 274, 275. The data input interface 271 may be connected to an extractor module 260 configured to extract data for a certificate to verify.

[0154] The key input interface 272 may be connected to the output interface 253 of the key derivation module 250. It is configured to receive a secret key (i.e., a symmetric key). The encryption block 273 may be configured to execute an authenticated encryption algorithm to encrypt the input data extracted from the certificate and generate an authentication tag from this input data, with the secret key received through the key input interface 272. This authenticated encryption algorithm may be the same as the one executed by the AE module 170 of the first computing device 100. The first output interface 274 may be configured to output this authentication tag. The second output interface 275 may be configured to output the encrypted data.

[0155] The verification engine 290 may include a certificate verification module 280 configured to verify each certificate of a chain of certificates, based on a comparison between an authentication tag extracted from said certificate and a verification tag generated by the second computing device 200, as will be described in more detail in the description of the method. The certificate verification module 280 may be connected in input to the extractor 160 and to the output interface 275 of the AE module 270.

[0156] An attestation method for attesting authenticity of the first computing device 100 will now be described, according to an embodiment, with reference to FIGS. 6 to 15.

[0157] FIG. 6 represents a flow chart of an attestation method 1000 for attesting device's authenticity, by which the first computing device 100 may provide verifiable evidence of its authenticity to the second computing device 200.

[0158] It should be appreciated by those skilled in the art that any functions, engines, block diagrams, flow diagrams, state transition diagrams, flowchart and/or data structures described herein represent conceptual views of illustrative circuitry embodying the principles of the invention. Similarly, it will be appreciated that any flow charts, flow diagrams, state transition diagrams, pseudo code, and the like represent various processes.

[0159] Although a flow chart may describe a set of steps as a sequential process, many of the steps may be performed in parallel, concurrently, or simultaneously. Also, some steps may be omitted, combined or performed in different order. A process may be terminated when its steps are completed but may also have additional steps not disclosed in the figure or description.

[0160] Each described process, function, engine, block, step described herein can be implemented in hardware, software, firmware, middleware, microcode, or any suitable combination thereof.

[0161] When implemented in software, firmware, middleware or microcode, instructions to perform the necessary tasks may be stored in memory means that may be or not included in a host device or host system configured to execute the instructions. The instructions may be transmitted over a computer-readable medium and be loaded onto the host device or host system. The instructions are configured to cause the host device or host system to perform one or more functions disclosed herein. For example, as mentioned above, at least one memory may include or store instructions, the at least one memory and the instructions may be configured to, with at least one processor, cause the host device or host system to perform the one or more functions.

[0162] The attestation method 1000 may include a provisioning process 1100 for provisioning the first computing device 100 with a unique device secret UDS and a provisioning process 1150 for provisioning the second computing device 200 with at least one UDS-derived key Ka. The provisioning processes 1100, 1150 may be performed by a provisioning infrastructure 400.

[0163] The provisioning processes 1100, 1150 are illustrated in more detail in FIGS. 7A and 7B, according to an embodiment.

[0164] The provisioning process 1100 may include a step 1110 of generating the UDS and uniquely assigning the UDS to the first computing device 100. For example, the UDS may be a random or pseudo-random value. Then, in a following step 1120, the UDS may be securely stored in the first computing device 100, for example during a manufacturing process in a secure environment. The UDS may be embedded in the device's hardware. The UDS may be associated with a UDS unique identifier, UDS_ID. The UDS identifier UDS_ID may be generated upon generation of the UDS, and/or it may correspond to a device unique identifier uniquely assigned to the first computing device 100. This unique identifier UDS_ID can be stored in the first computing device 100, for example in association with the UDS. After provisioning 1100, this UDS may not be accessible or modifiable by mutable software in the computing device 100, to avoid any leakage of the UDS out of the computing device 100.

[0165] The provisioning process 1100 may further include a step 1130 of provisioning the first computing device 100 with at least one higher-level input value Cst_Auth and an initial value Cst_Init for the lower-level input 153 of the key ladder 150. For example, the higher-level input value Cst_Auth and the initial value Cst_Init are random or pseudo-random values, that may be different, both generated by the provisioning infrastructure 400. The higher-level input value Cst_Auth and the initial value Cst_Init can be stored in the memory means 120 of the first computing device 100.

[0166] In a system including a plurality of first computing devices 100, the UDS may be different for each first computing device 100 but the pair of input values Cst_Auth, Cst_Init may be global input values that are the same for all the first computing devices 100. Alternatively, different pairs of input values could be used for the different first computing devices 100 respectively.

[0167] The step 1120 of provisioning the first computing device 100 with the UDS may preferably be performed at device manufacturing time (i.e., when the first computing device 100 is manufactured). Advantageously, the UDS can never be changed. The step 1130 of provisioning the first computing device 100 with the values Cst_Auth, Cst_Init may be performed at manufacturing time and/or later, at any time, and can be changed or updated for example with a software update.

[0168] The provisioning process 1150 allows to provision the second computing device 200 with the UDS-derived key Ka derived from the UDS that has been pre-generated and assigned to the first computing device 100, and with the initial value Cst_Init. The provisioning process 1150 can be advantageously performed by the provisioning infrastructure 400 holding the UDS at least during the provisioning. The provisioning process 1150 may include a step 1160 of deriving the key Ka from the UDS with the predetermined input value Cst_Auth. This key Ka corresponds to (i.e., is

equal to or the same as) the higher-level key or intermediate key K_2 that will be derived in operation by the key ladder 150 of the first computing device 100, from the UDS used as a key with the higher-level input value Cst_Auth. In the step 1160, the key Ka may be derived from the UDS by performing the same cryptographic operation(s) as the one (s) that will be carried out by the higher-level key derivation module 154 of the key ladder 150 in operation. The provisioning process 1150 may further include a step 1170 of transmitting the key Ka, its associated UDS identifier UDS_ID, and of storing said key Ka and said associated UDS identifier UDS_ID in the memory means 220 of the second computing device 200. The provisioning process 1150 may further include a step 1180 of transmitting the initial value Cst_Init to the second computing device 200, for example through the network 300, and of storing said initial value Cst_Init in the memory means 220 of the second computing device 200. The steps 1170 and 1180 may be combined.

[0169] The steps 1170 and 1180 of provisioning the second computing device 200 with the Ka and Cst_Init may be performed at manufacturing time and/or later, at any time, and can be changed or updated for example with a software update.

[0170] After the provisioning processes 1100, 1150, the provisioning infrastructure 400 that has pre-generated the UDS may not retain this UDS, for security reasons. The UDS of the first computing device 100 may be deleted from the provisioning infrastructure 400 and may thus be securely held only in the first computing device 100. However, in case it is desired to change or update Cst_Auth and Ka over time, the UDS may be securely kept, for example in a very high security basement, and be only used if it is needed to generate a new Ka with a new Cst_Auth.

[0171] With reference to FIG. 6, the attestation method 1000 may include a step 1200 of transmitting a request for attesting authenticity from the second computing device 200 to the first computing device 100.

[0172] Upon reception of this request, the first computing device 100 may generate a plurality of chained certificates (i.e., a chain of certificates) in a step 1300, and transmit the chain of certificates to the second computing device 200 in a step 1400. The chained certificates may be transmitted all together or could be transmitted successively and/or separately.

[0173] However, the step 1200 of transmitting a request for attesting authenticity is optional. The step 1300 of generating a chain of certificates may be carried out by the first computing device 100 on its own initiative, or upon reception of a request from a third party, or upon occurrence of a predetermined event.

[0174] The attestation method 1000 may further include a step 1500 of verifying the validity of the chained certificates, carried out by the second computing device 200.

[0175] Optionally, the second computing device 200 may transmit challenge data CH_D to the first computing device 100, to prevent anti-replay attacks. The challenge data CH_D may for example include a cryptographic nonce (i.e., a number randomly or pseudo-randomly generated by the second device 200). The challenge data CH_D may be embedded in the request for attesting authenticity transmitted from the second computing device 200 to the first computing device 100 in the step 1200. Alternatively, it may be transmitted separately. The challenge data (e.g., a nonce) may be issued in the attestation method 1000 to ensure that

an old chain of certificates cannot be reused in a replay attack. In an embodiment, the challenge data CH_D (e.g., a nonce) is transmitted from the second computing device 200 to the first computing device 100 in association with an expiration time (for example 1 minute) and optionally the identity of the first computing device 100. Thus, the first computing device 100 has to provide the chain of certificates associated with this challenge-data CH_D (e.g., the nonce) within the expiration time. Otherwise, the attestation would be refused.

[0176] The chain of certificates generated in the step 1300 may include a plurality of N+1 chained certificates Certificate_i with i from 1 to N+1 and N>1. The N+1 chained certificates may include:

[0177] N chained certificates Certificate_i, with i from 1 to N, respectively related to or associated with N sets of device-specific data D_i, and

[0178] optionally a final or last or bottom N+1th chained certificate for anti-replay purpose.

[0179] The generation step 1300 may include successively executing N certificate generation processes 1300_{L_i}, or generation processes, each certificate generation process 1300_{L_i} allowing to generate one certificate

[0180] Certificate_i of the N chained certificates. The N certificate generation processes 1300_{L_i} may thus be layered (i.e., successively executed in layers L₁, L₂, . . . , L_N).

[0181] With reference to FIG. 8, the first or top certificate generation process 1300_{L₁} (i.e., for layer L₁) may include the following steps carried out by the key ladder 150 of the first computing device 100:

[0182] in a step 1310_{L₁}, receiving the higher-level input value Cst_Auth through the input interface 152, and deriving, by the higher-level KDF module 154 (e.g., a decryption module), a higher-level symmetric key K₂, that is an intermediate key, from the UDS used as a key with the input value Cst_Auth;

[0183] in a step 1320_{L₁}, receiving the lower-level input value Cst_Init through the input interface 153, and deriving, by the lower-level KDF module 155 (e.g., a decryption module), a lower-level symmetric key K_{1,L1}, that is a final derived key, from the intermediate key K₂ used as a key with the input value Cst_Init;

[0184] in a step 1330_{L₁}, outputting from the key ladder 150 the lower-level symmetric key K_{1,L1} through the output interface 156.

[0185] The secret UDS and the input values Cst_Auth and Cst_Init are obtained or read in the memory means 120 of the first computing device 100 and inputted to the key ladder 150 in the steps 1310_{L₁} and 1320_{L₁}.

[0186] The first or top layer L_i certificate generation process 1300_{L_i} may further include a step 1340_{L_i} of determining or measuring a first set of device-specific data D₁. This step 1340_{L_i} can be carried out by the measurement module 160.

[0187] By definition, a set of device-specific data D₁, D₂, . . . is specific to the first computing device 100. It allows to provide evidence of the authenticity of the first computing device 100 to another party, for example to the second computing device 200 or any other party that the first computing device 100 runs the correct hardware and software, based on evidence. The evidence may include hardware attributes like configuration(s), state(s), identifier(s) such as an UID (Unique Identifier), etc., and software

attributes like version(s), code hash(es), etc. These items of evidence may be based on cryptography using unique keys of the first computing device 100. For example, the set of device-specific data allows the second computing device 200 or other party to validate at least an operating state of the first computing device 100. It may also allow the second computing device 200 or other party to validate the identity of the first computing device. However, it is emphasized that the use of the UDS to generate the chain of certificates allows to prove the identity of the first computing device 100.

[0188] The set of device-specific data can be self-measured, or self-determined, by the first computing device 100 (here by the measurement module 160), for example when executing the generation process 1300_{L_i}. It can represent a software state of the first computing device 100 (e.g., versions, hashes of software components, . . .), and/or a hardware state of the first computing device 100 (e.g., debug configuration, hardware fusing configuration, . . .), or a combination of a software state and a hardware state of the first computing device 100.

[0189] In an embodiment, the set of device-specific data may include time information such as timestamp and/or validity period of the hardware and/or software state.

[0190] The set of device-specific data may include a collection of data elements (e.g., digital values) respectively representing information of different predetermined types related to the first computing device 100. Each data element may for example represent a property, an operating parameter or state, and/or a component (e.g., a software component or a hardware component) of the first computing device 100. In an embodiment, each type of data element may have a predetermined fixed length. In an embodiment, at least part of the data elements of the set of device-specific data may be hashed, for example to be mapped to predetermined fixed-length (or fixed-size) values. The data elements may be hashed in a predetermined (given) order to obtain the set of device-specific data D₁, D₂, etc. Thus, the set of device-specific data may include the data itself or a hash of the data or a mix of the data and a hash of the data. In any case, it should be clear for the second computing device 200 how to recompute the authentication tag.

[0191] In an illustrative and non-limitative embodiment, the set of device-specific data D_i with 1≤i≤N may include a collection of DICE input values as specified in the specifications Open Profile for DICE v2.5. These input values can include code (64 bytes), configuration data (64 bytes), authority data (64 bytes), mode decision (1 byte), hidden inputs (64 bytes). In accordance with the DICE specifications, the input values can be hashed in this order: code, configuration data, authority data, mode decision, hidden inputs. The hashed input values can be referred by “H_i (code+config+authority+mode+hidden)”. For example, a boot procedure of the first computing device 100 may include different stages carried out by executing different software components, respectively. The software component of each stage of the boot procedure may be hashed and used by the key ladder 150. The certification chain can thus attest that the first computing device 100 performed the boot procedure correctly using the correct software and hardware.

[0192] However, the present disclosure is not limited to the DICE input values and applies more generally to any set of device-specific data as above defined.

[0193] In a step 1350_L₁, the first measured or determined set of device-specific data D_i (e.g., H_i(code+config+authority+mode+hidden)) is provided as input to the AE module 170.

[0194] In a step 1360_L₁, the first computing device 100 encrypts the set of

[0195] device-specific data D_i (e.g., H₁(code+config+authority+mode+hidden)) and generates an authentication tag TAG₁ from the set of device-specific data D₁, by executing the algorithm of authenticated encryption using the derived symmetric or secret key K_{1,L1} as an encryption key and the set of device-specific data D_i as input data. The expression E_{K_{1,L1}}(D₁) refers to encryption of the set of device-specific data D_i with the derived key K_{1,L1} serving as cryptographic key. The step 1360_L₁ can be carried out by the AE module 170.

[0196] In a step 1370_L₁, the first computing device 100 generates a first or top certificate Certificate_1. This first certificate Certificate_1 can include the following data:

[0197] the UDS identifier UDS_ID of the first computing device 100;

[0198] the first set of device-specific data D1 (e.g., H₁(code+config+authority+mode+hidden));

[0199] the authentication tag TAG₁.

[0200] The UDS_ID may be obtained from the memory means 120 of the first computing device 100. The set of device-specific data D_i may be provided by the measurement module 160. The authentication tag TAG₁ may be provided by the AE module 170.

[0201] The attestation method 100 may further comprise a step 1380_L₁ of providing the encrypted set of device-specific data E_{K_{1,L1}}(D₁) for the subsequent certificate generation process (i.e., for the next or following certificate generation process), as input to the key ladder 150. More precisely, the encrypted first set of device-specific data E_{K_{1,L1}}(D₁) is transmitted to the lower-level input interface 153 of the key ladder 150 for the subsequent certificate generation process, here for the second certificate generation process 1300_L₂.

[0202] In the step 1360_L₁, the first computing device 100 derives two distinct elements from the set of device-specific data D₁:

[0203] TAG₁, the authentication tag, and

[0204] E_{K_{1,L1}}(D₁), the encrypted set of device-specific data D₁.

[0205] The first element, TAG₁, is used to authenticate the certificate Certificate_1 of the current layer L₁. The second element, E_{K_{1,L1}}(D₁), serves as input data for the key ladder in the next layer L₂.

[0206] After executing the first certificate generation process 1300_L₁, each subsequent certificate generation process 1300_L_i, from the second certificate generation process to the Nth certificate generation process (i.e., at each layer i, with 1<i≤N), includes the steps 1310_L_i to 1380_L_i described below and illustrated in FIG. 9.

[0207] With reference to FIG. 9, the ith certificate generation process 1300_L_i with 1<i≤N is similar or analogous to the first certificate generation process 1300_L₁, but only differ from it by some features that will be apparent in the description given below.

[0208] In the step 1310_L_i, the key ladder 150 receives the higher-level input value Cst_Auth through the input interface 152, and derives, through the higher-level KDF module

154 (e.g., a decryption module), the symmetric key K₂ from the UDS used as a key with this input value Cst_Auth.

[0209] In the step 1320_L_i, the key ladder 150 receives as lower-level input the previous encrypted set of device-specific data E_{K_{1,Li-1}}(D_{i-1}) (i.e., the encrypted set of device-specific data produced by the preceding generation process), and derives, through the lower-level KDF module 155 (e.g., a decryption module), a symmetric key K_{1,Li} (namely, a lower-level symmetric key) from the intermediate or higher-level key K₂ used as a key with this lower-level input value E_{K_{1,Li-1}}(D_{i-1}).

[0210] In the step 1330_L_i, the key ladder 150 outputs the symmetric key K_{1,Li}.

[0211] In the step 1340_L_i, the first computing device 100 determines or measures a ith set of device-specific data D_i (e.g., H_i(code+config+authority+mode+hidden)). The successive sets of device-specific data D₁, D₂, . . . , D_N may be different from one another. For example, in H_i(code+config+authority+mode+hidden), at least the code data is different.

[0212] In the step 1350_L_i, the first computing device 100 inputs the set of device specific data D_i (e.g., H_i(code+config+authority+mode+hidden)) into the AE module 170.

[0213] In the step 1360_L_i, the first computing device 100 encrypts the set of device-specific data D_i (e.g., H_i(code+config+authority+mode+hidden)) to produce the encrypted set of device-specific data E_{K_{1,Li}}(D_i) and generates an authentication tag TAG_i from the set of device-specific data D_i, by executing the algorithm of authenticated encryption using the derived symmetric or secret key K_{1,Li} as an encryption key and the set of device-specific data D_i as input data. This step 1360_L_i may be performed by the AE module 170.

[0214] In the step 1370_L_i, the first computing device 100 generates the ith certificate Certificate_i including the UDS identifier UDS_ID of the first computing device 100, the set of device-specific data D_i (e.g., H_i(code+config+authority+mode+hidden)), and the authentication TAG_i. The step 1370_L_i may be performed by the generator 180.

[0215] In the step 1380_L_i, the encrypted set of device-specific data E_{K_{1,Li}}(D_i) is provided for the subsequent certificate generation process (i.e., for the next or following certificate generation process at layer L_{i+1}), as input to the key ladder 150, the encrypted set of device-specific data E_{K_{1,Li}}(D_i) being transmitted to the lower-level input interface 153 of the key ladder 150 to be used as lower-level input value in the subsequent certificate generation process, at layer L_{i+1}.

[0216] In the step 1360_L_i, the first computing device 100 derives two distinct elements from the set of device-specific data D_i:

[0217] TAG_i, the authentication tag, and

[0218] E_{K_{1,Li}}(D_i), the encrypted set of device-specific data D_i.

[0219] The first element, TAG_i, is used to authenticate the certificate Certificate_i of the current layer L_i. The second element, E_{K_{1,Li}}(D_i), serves as input data for the key ladder in the next layer, L_{i+1}.

[0220] Optionally, after executing the N certificate generation processes 1300_L_i with 1≤i≤N, the attestation method may further include a N+1th certificate generation process 1300_L_{N+1} of generating a N+1th or final certificate including the challenge data CH_D (e.g., a nonce value) received by the first computing device 100 from the second computing device 200.

[0221] The $N+1^{th}$ generation process $1300_{L_{N+1}}$ may include the steps $1310_{L_{N+1}}$ to $1380_{L_{N+1}}$ described below and illustrated in FIG. 10.

[0222] The $N+1^{th}$ certificate generation process $1300_{L_{N+1}}$ is similar or analogous to any certificate generation process at layer L_i with $1 \leq i \leq N$, but only differ from it by some features that will be apparent in the description below.

[0223] In the $N+1^{th}$ certificate generation process $1300_{L_{N+1}}$, the first computing device **100** embeds or includes the challenge data CH_D in the $N+1^{th}$ certificate, and the authentication tag TAG_{N+1} is generated from the challenge data CH_D , instead of a set of device-specific data.

[0224] In the step $1310_{L_{N+1}}$, the key ladder **150** receives the higher-level input value Cst_Auth through the input interface **152**, and derives, through the higher-level KDF module **154** (e.g., a decryption module), the symmetric key K_2 from the UDS used as a key with the input value Cst_Auth .

[0225] In the step $1320_{L_{N+1}}$, the key ladder **150** receives, through the lower-level input interface **153**, the encrypted set of device-specific data $E_{K_{1,LN}}(D_N)$ produced by the N^{th} certificate generation process as lower-level input, and derives, through the lower-level KDF module **155** (e.g., a decryption module), a symmetric key $K_{1,LN+1}$ (namely, a lower-level symmetric key) from the intermediate key K_2 used as a key with this lower-level input value $E_{K_{1,LN}}(D_N)$.

[0226] In the step $1330_{L_{N+1}}$, the key ladder **150** outputs the symmetric key $K_{1,LN+1}$.

[0227] In the step $1350_{L_{N+1}}$, the first computing device **100** inputs the challenge data CH_D , received from the second computing device **200** for example in the step **1200**, into the AE module **170** as input data to encrypt. The $N+1^{th}$ certificate generation process may not include a step of measuring or determining a set of device-specific data.

[0228] In the step $1360_{L_{N+1}}$, the AE module **170** generates an authentication tag TAG_{N+1} from the challenge data CH_D , by executing the algorithm of authenticated encryption using the derived symmetric or secret key $K_{1,LN+1}$ as an encryption key and the challenge data CH_D as input data. In the present embodiment, the AE module **170** may also encrypt the challenge data CH_D , when executing the authenticated encryption algorithm, but the encrypted challenge data may not be retained and/or used by the first computing device **100**.

[0229] Then, in the step $1370_{L_{N+1}}$, the first computing device **100** generates the $N+1^{th}$ certificate $Certificate_{N+1}$ including the UDS identifier UDS_ID of the first computing device **100**, the challenge data CH_D , and the authentication tag TAG_{N+1} .

[0230] It should be noted that the higher-level input value Cst_Auth received by the key ladder **150** through the higher-level input interface **152** is a predetermined constant that is the same for all the certificate generation processes 1300_{L_i} at layer L_i with $1 \leq i \leq N+1$.

[0231] As previously described, the $N+1$ chained certificates are transmitted from the first computing device **100** to the second computing device **200**, in the step **1400**.

[0232] FIG. 11 shows an overview of the attestation method carried out by the first computing device **100** to generate the chained certificates $Certificate_1$, $Certificate_2$, ..., $Certificate_{N+1}$, according to the first Embodiment.

[0233] A single key ladder **150** may be used to derive the multiple keys K_2 and $K_{1,Li}$ for any layer L_i with $1 \leq i \leq N+1$. This key ladder **150** may be similarly configured for all

layers L_i with $1 \leq i \leq N+1$ so as to execute the same cryptographic algorithms using the same UDS and Cst_Auth , as previously described.

[0234] In another embodiment, the single key **150** may be configured differently for different layers L_i with $1 \leq i \leq N+1$, for example to execute different cryptographic algorithms and/or use different UDS and/or input value like Cst_Auth .

[0235] In another embodiment, the first computing device **100** may include a plurality of key ladders (i.e., different key ladders). For example, the first computing device **100** may include $N+1$ key ladders. In that case, a different key ladder may be used to derive the keys K_2 and $K_{1,Li}$ at each layer L_i with $1 \leq i \leq N+1$.

[0236] In any case, the second computing device **200** should be configured in a similar manner to the first computing device **100** to be able to perform similar computations.

[0237] In a step **1450**, the second computing device **200** may receive the $N+1$ chained certificates $Certificate_i$ with $1 \leq i \leq N+1$ transmitted by the first computing device **100**.

[0238] As previously mentioned, each certificate $Certificate_i$, for $1 \leq i \leq N+1$, may include:

[0239] a UDS identifier, UDS_ID ;

[0240] for $1 \leq i \leq N$, a set of device-specific data D_i (e.g., $H_i(\text{code}+\text{config}+\text{authority}+\text{mode}+\text{hidden})$);

[0241] for $i=N+1$, the challenge data CH_D ; and

[0242] an authentication tag TAG_i .

[0243] In the final stage, in other words for $i=N+1$, the data used as input of the key ladder **150** could be the challenge data CH_D (e.g., a nonce) as above described. Alternatively, this input data of the key ladder **150** in the final stage (for $i=N+1$) could be a hash of the challenge data CH_D or any other data, or hash of data, as long as it is clear for the second computing device **200** how to recompute the authentication tag TAG_{N+1} .

[0244] The certificate $Certificate_i$ may include additional information such as expiration information, validity time information, application specific data (e.g., symmetric or asymmetric keys or nonce or any local application data that needs to be sent to the second computing device **200** and linked to the chain of trust provided through the remote attestation), etc.

[0245] In the step **1500**, the second computing device **200** verifies the validity of the $N+1$ chained certificates. This verification step **1500** may be layered, and include successively executing $N+1$ certificate verification processes 1500_{L_i} , with $1 \leq i \leq N+1$, of verifying one certificate of the $N+1$ chained certificates $Certificate_i$. Each certificate verification process 1500_{L_i} includes the steps, carried out by the second computing device **200**, as described below and with reference to FIGS. 12, 13 and 14.

[0246] As shown in FIG. 12, the first or top certificate generation process 1500_{L_1} , includes a step 1510_{L_1} , of:

[0247] retrieving from the memory means **220** the key K_a that is associated with the UDS identifier UDS_ID included in the certificate $Certificate_1$,

[0248] receiving, by the key derivation module **250**, the initial input value Cst_Init (stored in the memory means **220**) through the input interface **252**, and

[0249] deriving, through the key derivation module **250**, a symmetric key $K_{1,L1}$ from the symmetric key K_a used as a key with the initial input value Cst_Init .

[0250] In an embodiment, the key derivation module **250** may execute the same cryptographic algorithm(s) (e.g., a

decryption algorithm) as the one(s) executed by the lower-level module **155** of the key ladder **150** of the first computing device **100**. The key K_a and the initial input value Cst_Init may be obtained in the memory means **220** of the second computing device **200**. As previously indicated, they may have been provisioned in the second computing device **200** during the provisioning process **1150**.

[0251] In the step **1520**_{L₁}, the AE module **270** of the second computing device **200** encrypts the set of device-specific data D_i (e.g., $H_i(\text{code}+\text{config}+\text{authority}+\text{mode}+\text{hidden})$), extracted from the first or top certificate **Certificate_1**, and generates a verification tag TAG'_1 from this set of device-specific data D_i , using the K_a -derived symmetric key $K'_{1,L1}$ as a key. The encryption of the set of device specific data D_i with the K_a -derived symmetric key $K'_{1,L1}$ is referred as $E_{K'_{1,L1}}(D_1)$.

[0252] In the step **1530**_{L₁}, the certificate verification module **280** compares the verification tag TAG'_1 generated by the second computing device **200** in the step **1520**_{L₁} and the authentication tag TAG_1 extracted from the certificate **Certificate_1** to verify if they match.

[0253] If the verification tag TAG'_1 generated by the second computing device **200** and the authentication tag TAG_1 extracted from the certificate **Certificate_1** match (i.e., if they are identical), the second computing device **200** validates the certificate **Certificate_1**, in a step **1550**_{L₁}.

[0254] If the verification tag TAG'_1 and the authentication tag TAG_1 do not match, the second computing device **200** denies (i.e., does not validate) the certificate **Certificate_1** and consequently denies (i.e., does not validate) the authenticity of the first computing device **100**, in a step **1540**_{L₁}. The verification method **1500** is ended. Optionally, the second computing device **200** may generate a non-validation message indicating that the authenticity of the first computing device **100** has been denied. This message may be conveyed to a user through the user interface **240** of the second computing device **200** and/or transmitted to the first computing device **100** through the network **300**.

[0255] If the certificate **Certificate_1** has been successfully validated, the second computing device **200** may provide the encrypted set of device-specific data $E_{K'_{1,L1}}(D_1)$ as an input to the key derivation module **250** for the subsequent certificate verification process, here **1500**_{L₂}, in a step **1560**_{L₁}.

[0256] After executing the first certificate verification process **1500**_{L₁}, each subsequent certificate verification process, from the second certificate verification process to the N^{th} certificate verification process (i.e., at each layer i , with $1 < i \leq N$), includes the steps **1510**_{L_i} to **1560**_{L_i} described below and illustrated in FIG. 13.

[0257] The certificate verification process **1500**_{L_i} is similar or analogous to the first certificate verification process **1500**_{L₁}, but only differ from it by some features that will be apparent in the description below.

[0258] In the step **1510**_{L_i}, the second computing device **200**:

[0259] retrieves from the memory means **220** the key K_a that is associated with the UDS identifier UDS_ID included in the certificate **Certificate_i**,

[0260] receives, by the key derivation module **250**, the input value $E_{K'_{1,L1}}(D_{i-1})$ through the input interface **252**, and

[0261] derives, through the key derivation module **250**, a symmetric key $K'_{i,Li}$ from this symmetric key K_a used as a key with the input value $E_{K'_{1,L1}}(D_{i-1})$.

[0262] The key K_a may be obtained from the memory means **220** of the second computing device **200**. The input value $E_{K'_{1,L1}}(D_{i-1})$ has been produced in the preceding certificate verification process **1500**_{L_{i-1}}. In an embodiment, in the step **1510**_{L_i}, the key derivation module **250** may execute the same cryptographic algorithm(s) (e.g., a decryption algorithm) as the one(s) executed by the lower-level module **155** of the key ladder **150** of the first computing device **100**.

[0263] In the step **1520**_{L_i}, the AE module **270** of the second computing device **200** may encrypt the set of device-specific data D_i (e.g., $H_i(\text{code}+\text{config}+\text{authority}+\text{mode}+\text{hidden})$), extracted from the certificate **Certificate_i**, and generate a verification tag TAG'_i from this set of device-specific data D_i , using the K_a -derived symmetric key $K'_{i,Li}$ as a key. The encryption of the set of device specific data D_i with the K_a -derived symmetric key $K'_{i,Li}$ is referred as $E_{K'_{i,Li}}(D_i)$. In the step **1530**_{L_i}, the certificate verification module **280** compares the

[0264] verification tag TAG'_i generated by the second computing device **200** in the step **1520**_{L_i} and the authentication tag TAG_i extracted from the certificate **Certificate_i** to verify if they match (i.e., if they are identical).

[0265] If the verification tag TAG'_i generated by the second computing device **200** and the authentication tag TAG_i extracted from the certificate **Certificate_i** match, the second computing device **200** validates the certificate **Certificate_i**, in a step **1550**_{L_i}.

[0266] If the verification tag TAG'_i and the authentication tag TAG_i do not match, the second computing device **200** denies (i.e., does not validate) the certificate **Certificate_i** and consequently denies (i.e., does not validate) the authenticity of the first computing device **100**, in a step **1540**_{L_i}. The verification method **1500** is ended. The second computing device **200** may generate a non-validation message indicating that the authenticity of the first computing device **100** has been denied. This message may be conveyed to a user through the user interface **240** of the second computing device **200** and/or transmitted to the first computing device **100** through the network **300**.

[0267] If the certificate **Certificate_i** has been successfully validated, the second computing device **200** may provide the encrypted set of device-specific data $E_{K'_{i,Li}}(D_i)$ for the subsequent certificate verification process **1500**_{L_{i+1}} as an input to the key derivation module **250**, here **1500**_{L_{i+1}}, in a step **1560**_{L_i}.

[0268] Optionally, after executing the N certificate verification processes **1500**_{L_i} with $1 \leq i \leq N$, the attestation method carried out by the second computing device **200** may further include a $N+1^{th}$ certificate verification process **1500**_{L_{N+1}} of verifying the validity of the $N+1^{th}$ or final or bottom certificate **Certificate_{N+1}**, that is an anti-replay certificate. This final certificate **Certificate_{N+1}** may include the challenge data CH_D (e.g., a nonce value) transmitted by the second computing device **200** from the first computing device **100**.

[0269] As shown in FIG. 14, the $N+1^{th}$ verification process **1300**_{L_{N+1}} may include the steps **1510**_{L_{N+1}} to **1560**_{L_{N+1}} described below.

[0270] The $N+1^{th}$ certificate verification process **1500**_{L_{N+1}} may be similar or analogous to any certificate verification process **1500**_{L_i} at layer L_i with $1 < i \leq N$, but only differ from it by some features that will be apparent in the description below.

[0271] In the $N+1^{th}$ or last certificate verification process **1500**_{L_{N+1}}, the second computing device **200** verifies if the challenge data CH_D included in the $N+1^{th}$ or last certificate Certificate_N+1 is not replayed and, in a positive event only, it generates a verification tag TAG'_{N+1} from this challenge data CH_D, and the authentication tag TAG_{N+1} and compares it with the authentication tag TAG_{N+1} included in the final certificate Certificate_N+1. In an embodiment, in case the challenge data CH_D (e.g., a nonce) was transmitted from the second device **200** to the first device **100** in association with an expiration time (e.g., an expiration date) and the first computing device's identity, the second computing device **200** may verify if the first computing device **100** has provided the chain of certificates associated with the challenge data CH_D (e.g., the nonce) before this expiration time or date has expired. If the expiration time has already expired, the attestation with the challenge_data (e.g., nonce) is refused. The attestation can only be accepted if the chain of certificates is received by the second computing device **200** before expiration of the expiration time. Optionally, after expiration of the expiration time, a new challenge data (e.g., a new nonce) can be generated.

[0272] More precisely, in the step **1510**_{L_{N+1}}, the second computing device **200**:

[0273] retrieves from the memory means **220** the key Ka that is associated with the UDS identifier UDS_ID included in the certificate Certificate_N+1,

[0274] receives, by the key derivation module **250**, the input value $E_{K'_{1,LN}}(D_N)$ that has been produced in the preceding certificate verification process **1500**_{LN}, through the input interface **252**, and

[0275] derives, through the key derivation module **250**, a symmetric key $K'_{1,LN+1}$ from this symmetric key Ka used as a key with the input value $E_{K'_{1,LN}}(D_N)$.

[0276] In a step **1515**_{L_{N+1}}, the second computing device **200** may verify if the challenge data CH_D (e.g., a nonce) included in the certificate Certificate_N+1 is not replayed (i.e., if it is used for the first time in a chain of certificates).

[0277] In a negative event, in other words if the challenge data CH_D included in the certificate Certificate_N+1 has already been played or used in another chain of certificates, the authenticity of the first computing device **100** is denied in a step **1516**_{L_{N+1}}, and the attestation method is ended.

[0278] In a positive event, if the challenge data CH_D included in the certificate Certificate_N+1 has not already been played or used in another chain of certificates, the process goes to the next step **1520**_{L_{N+1}} of generating a verification tag TAG'_{N+1} from the challenge data CH_D extracted from the certificate Certificate_N+1. The generation of the verification tag TAG'_{N+1} is performed by the AE module **270** of the second computing device **200**. In the present embodiment, the AE module **270** may also encrypt the challenge data CH_D, when executing the authenticated encryption algorithm, but the encrypted challenge data $E_{K'_{1,LN+1}}(CH_D)$ may not be retained and/or used by the second computing device **200**.

[0279] In the step **1530**_{L_{N+1}}, the certificate verification module **280** compares the verification tag TAG'_{N+1} generated by the second computing device **200** in the step **1520**_{L_{N+1}} and the authentication tag TAG_{N+1} extracted from the final certificate Certificate_N+1 to verify if they match.

[0280] If the tag TAG'_i does not match the tag TAG_i, the second computing device **200** denies (i.e., does not validate)

the certificate Certificate_i and consequently denies (i.e., does not validate) the authenticity of the first computing device **100**, in a step **1540**_{L_{N+1}}. The verification method **1500** is ended. The second computing device **200** may generate a non-validation message indicating that the authenticity of the first computing device **100** has been denied. This message may be conveyed to a user through the user interface **240** of the second computing device **200** and/or transmitted to the first computing device **100** through the network **300**.

[0281] If the tag TAG'_{N+1} matches the tag TAG_{N+1}, the second computing device **200** validates the certificate Certificate_N+1, in a step **1550**_{L_{N+1}}.

[0282] Furthermore, after the last successful certificate validation in the step **1550**_{L_{N+1}}, as all the chained certificates Certificate_i from i=1 to i=N+1 have been successfully validated, the second computing device **200** validates the authenticity of the first computing device **100**, in the step **1560**_{L_{N+1}}.

Second Embodiment

[0283] The second Embodiment is based on the first Embodiment and only differs from the first Embodiment by the features described below.

[0284] A main difference between the second Embodiment and the first Embodiment is that, in the second Embodiment, the attestation engine **190** of the first computing device **100** includes an encryption module and a tag generator, which are separate and/or distinct, instead of one authenticated encryption module **170** configured to simultaneously perform encryption and tag generation. In the second Embodiment, the encryption module and the tag generator may be coupled but configured to respectively use a first key K_{1,A,L_i} and a second key K_{1,B,L_i} , different from each other, to respectively encrypt a set of device-specific data D_i and generate the authentication tag TAG_i from said set of device-specific data D_i in the certificate generation process at any layer L_i with $1 \leq i \leq N$. For that purpose, the first computing device **100** may be provided or provisioned with two different higher-level input values Cst_Auth_A and Cst_Auth_B. The input values Cst_Auth_A and Cst_Auth_B may be stored in the memory means **120**.

[0285] As illustrative and non-limitative examples, the encryption module may be configured to perform a block cipher. This block cipher may be configured to operate in CBC (cipher Block Chaining) mode, or in CBC-CS1 (Cipher Block Chaining—CipherStealing 1) mode, or in CTR (Counter) mode. The tag generator may be configured to generate CMAC or HMAC use CBC/CBC-CS1/CTR encryption mode coupled with CMAC (block cipher-based message authentication code) or a HMAC (Hash-based message authentication code).

[0286] In this second Embodiment, the key ladder **150** of the first computing device **100** is configured to:

[0287] derive a first higher-level key $K_{2,A}$, through the higher-level module **154**, from the UDS used as a key with the first higher-level input value Cst_Auth_A, and

[0288] derive the lower level key K_{1,A,L_i} from $K_{2,A}$, through the lower-level module **155**, with the lower-level input that is either the initial value Cst_Init for the first or top layer L_1 , or the encrypted set of device-specific data $E_{K_{1,A,L_{i-1}}}(D_{i-1})$ provided by the preceding certificate generation process for each of the subsequent generation processes at layer L_i with $1 < i \leq N+1$.

[0289] Optionally, for the $N+1^{th}$ certificate generation process, the key ladder 150 may be configured not to derive a lower-level key $K_{1A, LN+1}$, as there is no need to perform data encryption with this key.

[0290] The key ladder 150 is also configured to:

[0291] derive a second higher-level key K_{2B} , through the higher-level module 154, from the UDS used as a key with the second higher-level input value Cst_Auth_B , and

[0292] derive the lower level key $K_{1B, Li}$ from K_{2B} , through the lower-level module 155, with the lower-level input that is either the initial value Cst_Init for the first or top layer L_i , or the encrypted set of device-specific data $E_{K_{1A, Li-1}}(D_{i-1})$ provided by the preceding certificate generation process for each of the subsequent generation processes at layer L_i with $1 < i \leq N+1$.

[0293] In the second Embodiment, in each certificate generation process at layer L_i with $1 \leq i \leq N$:

[0294] the key ladder 150 may derive the first symmetric key $K_{1A, Li}$ and the second symmetric key $K_{1B, Li}$, different from one another, from the UDS used as a key, respectively with Cst_Auth_A and Cst_Auth_B as higher-level input value;

[0295] the set of device-specific data D_i may be encrypted by the encryption module with the first symmetric key $K_{1A, Li}$ to produce $E_{K_{1A, Li}}(D_i)$; and

[0296] the authentication tag TAG_i may be generated, by the tag generator, from the set of device-specific data D_i with the second symmetric key $K_{1B, Li}$.

[0297] In the $N+1^{th}$ certificate generation process:

[0298] the key ladder 150 may derive the second symmetric key $K_{1B, LN+1}$ from the UDS used as a key with Cst_Auth_B as higher-level input value; and

[0299] the authentication tag TAG_{N+1} may be generated, by the tag generator, from the challenge data CH_D with the second symmetric key $K_{1B, LN+1}$.

[0300] In the $N+1^{th}$ certificate generation process, the encryption module does not need to encrypt data.

[0301] The key ladder 150 may use the same initial lower-level input value Cst_Init to derive the first key $K_{1A, L1}$ and the second key $K_{1B, L1}$, for example by changing the algorithm configured to derive these keys. Alternatively, the key ladder 150 may use two different lower-level input values Cst_Init_A and Cst_Init_B to respectively derive the first key $K_{1A, L1}$ and the second key $K_{1B, L1}$.

[0302] In the second Embodiment, the second computing device 100 may be provisioned with two UDS-derived keys K_a , K_b , both derived from the UDS of the first computing device 100. The key K_a may be derived from the UDS with the predetermined higher-level input value Cst_Auth_A , and the key K_b may be derived from the UDS with the predetermined higher-level input value Cst_Auth_B . These keys K_a , K_b respectively correspond to the first higher-level key K_{2A} and to the second higher-level key K_{2B} that will be derived, in operation, by the key ladder 150 of the first computing device 100 from its UDS used as a key respectively with the first higher-level input value Cst_Auth_A and the second higher-level input value Cst_Auth_B .

[0303] The verification engine 290 of the second computing device 100 may include an encryption module and a tag generator, separate and/or distinct from each other, instead of one authenticated encryption module 270 configured to simultaneously perform the encryption and tag generation. The encryption module and the tag generator are configured

to respectively use a first key $K'_{1A, Li}$ derived from K_a and a second key $K'_{1B, Li}$ derived from K_b to encrypt a set of device-specific data D_i and to generate the authentication tag

[0304] TAG_i from said set of device-specific data D_i in the certificate generation process at any layer L_i with $1 \leq i \leq N$. For that purpose, the second computing device 200 may be provided or provisioned with two different higher-level input values Cst_Auth_A and Cst_Auth_B . The input values Cst_Auth_A and Cst_Auth_B may be stored in the memory means 220.

[0305] In the second Embodiment, the key derivation module 250 is configured to derive a first key $K'_{1A, Li}$ from the key K_a with the lower-level input that is either the initial value Cst_Init for the first or top layer L_i , or the encrypted set of device-specific data $E_{K_{1A, Li-1}}(D_{i-1})$ provided by the preceding certificate verification process for each of the subsequent certificate verification processes at layer L_i with $1 < i \leq N+1$.

[0306] Optionally, for the $N+1^{th}$ certificate generation process, the key derivation module 250 may be configured not to derive a key $K'_{1A, LN+1}$, as there is no need to perform data encryption with this key.

[0307] The key derivation module 250 may also be configured to derive key $K'_{1B, Li}$ from the key K_b with the lower-level input that is either the initial value Cst_Init for the first or top layer L_i , or the encrypted set of device-specific data $E_{K_{1A, Li-1}}(D_{i-1})$ provided by the preceding certificate verification process for each of the subsequent certificate verification processes at layer L_i with $1 < i \leq N+1$.

[0308] In the second Embodiment, in each certificate verification process at layer L_i with $1 \leq i \leq N$:

[0309] the key derivation module 250 may derive the first symmetric key $K'_{1A, Li}$ and the second symmetric key $K'_{1B, Li}$, respectively from the symmetric keys K_a and K_b (pre-stored in memory);

[0310] the set of device-specific data D_i extracted from the certificate $Certificate_i$ may be encrypted by the encryption module with the first symmetric key $K'_{1A, Li}$ to produce $E_{K'_{1A, Li}}(D_i)$; and

[0311] the authentication tag TAG_i may be generated, by the tag generator, from the set of device-specific data D_i with the second symmetric key $K'_{1B, Li}$.

[0312] In the $N+1^{th}$ certificate verification process:

[0313] the key derivation module 250 may derive the second symmetric key $K'_{1B, LN+1}$ from the symmetric key K_b ; and

[0314] the authentication tag TAG_{N+1} may be generated, by the tag generator, from the challenge data CH_D extracted from the certificate $Certificate_N+1$ with this second symmetric key $K'_{1B, LN+1}$.

[0315] In the $N+1^{th}$ certificate verification process, the encryption module does not need to encrypt data.

[0316] According to the present disclosure, the attestation method, by which the first computing device 100 provides evidence of its authenticity to the second computing device 200, is implemented based on at least one key ladder. Many existing computing devices are already provided with a key ladder. Therefore, the present attestation method can be implemented on existing key ladders.

[0317] Many key ladders are using symmetric keys and symmetric cryptographic algorithms. However, a key ladder may be capable of using asymmetric key(s) and asymmetric cryptography. The present attestation method and system are not limited to symmetric keys and symmetric cryptography

but could be implemented based on asymmetric keys and asymmetric cryptography, or on a mix of symmetric keys and asymmetric keys and a mix of symmetric cryptography and asymmetric cryptography. The present authentication method and system could be implemented using different combinations of key derivation functions KDFs, that may be symmetric or asymmetric, and/or different cryptographic algorithms that may be symmetric or asymmetric.

[0318] The present attestation method allows validation of all the keys at all layers of the generation of the chain of certificates. If one key is comprised, for example changed by a false key, when generating the chain of certificates by the first computing device, this chain of certificates will not be validated by the second computing device and the authenticity of the first computing device will be denied. For example, even in case that an encrypted data set $E_{K_{1,i}}(D_i)$ used as input by the key ladder 150 is intercepted, nobody can simply implement the rest of the certificate generation without breaking the internal components of the key ladder 150 (for example, what exact combination of KDF and symmetric algorithm is performed by the key ladder 150) and the UDS key. The security of the present attestation scheme cannot be broken in case if key leakage or key compromise.

[0319] According to the present disclosure, the UDS of the first computing device does not need to be shared with the second computing device. The second computing device only needs to access one or more UDS-derived symmetric keys, for example K_a or (K_a, K_b) , and an initial value Cst_Init, to validate the chained certificates. These data elements are less sensitive data, and may be transmitted to the second computing device without security precautions or with less security precautions.

[0320] Furthermore, the present attestation method can be based on symmetric cryptography, which allows to achieve the attestation method very quickly and without consuming too much memory. It can also be quantum-resistant.

[0321] Other advantages of the present attestation method are given below:

[0322] it can be implemented on existing hardware by using existing key ladder;

[0323] the reporting process used to make the UDS-derived key K_a , and optionally the UDS-derived key K_b , available to the second computing device 200 is already in place, for example for DICE;

[0324] all the keys used to attest integrity or authenticity of the first computing device 100 can be validated;

[0325] the security of the present scheme cannot be broken if the USD is simply leaked;

[0326] the present scheme does not expose the USD on the second computing device.

[0327] The above advantages are not available in the DICE solution.

FINAL CONSIDERATIONS

[0328] Although an overview of the inventive subject matter has been described with reference to specific example embodiments, various modifications and changes may be made to these embodiments without departing from the broader spirit and scope of embodiments of the present invention. For example, various embodiments of features thereof may be mixed and matched or made optional by a person of ordinary skill in the art. Therefore, the Detailed Description is not to be taken in a limiting sense, and the

scope of various embodiments is defined only by the appended claims, along with the full range of equivalents to which such claims are entitled.

1. A computer-implemented method for attesting authenticity of a first computing device, said first computing device being provisioned with a unique device secret (“UDS”), the method comprising, by the first computing device:

generating an N chained certificate(s), the N chained certificate(s) attesting the authenticity of an N set(s) of device-specific data of the first computing device; and transmitting the N chained certificate(s) to a second computing device,

wherein the generating the N chained certificate(s) further includes a generation process that includes steps of:

- A) deriving, using a key ladder, a key from the UDS;
- B) generating an authentication tag from a set of device-specific data, using the key;
- C) generating a certificate, said certificate including the set of device-specific data and the authentication tag;
- D) encrypting the set of device-specific data with the key; and
- E) providing the encrypted set of device-specific data as an input to the key ladder for a subsequent generation process.

2. The computer-implemented method according to claim 1, wherein the key ladder has at least two levels and the deriving, using the key ladder, further includes:

- receiving a higher-level input value; and
- deriving a higher-level key from the UDS used as a key or from a UDS-derived key with the higher-level input value.

3. The computer-implemented method according to claim 2, wherein the higher-level input value received by the key ladder includes a predetermined constant that is the same for all the generation processes.

4. The computer-implemented method according to claim 2, wherein the deriving, using the key ladder, further includes:

- deriving the key from the higher-level key with a lower-level input value.

5. The computer-implemented method according to claim 4, wherein the deriving further includes

- for a first generation process, receiving, at the key ladder, a predetermined initial value as lower-level input value; and

- for each of subsequent generation processes, receiving, at the key ladder, the encrypted set of device-specific data provided by a preceding generation process as lower-level input value.

6. The computer-implemented method according to claim 1, wherein, the generating the authentication tag from the set of device-specific data, using the key and the encrypting the set of device-specific data with the key are combined by executing an authenticated encryption algorithm to simultaneously encrypt the set of device-specific data and generate the authentication tag from the set of device-specific data with the key derived from the UDS in the deriving.

7. The computer-implemented method according to claim 1, wherein the deriving further includes deriving, using the key ladder, two keys, different from one another, from the UDS used as a key,

- the generating the authentication tag further includes generating the authentication tag with one of the two keys, and

the generating the certificate further includes encrypting the set of device-specific data with the other of the two keys.

8. The computer-implemented method according to claim 7, wherein the two keys are separately derived from the UDS by the key ladder using respectively a first higher-level input value and a second higher-level input value, different from one another.

9. The computer-implemented method according to claim 1, wherein, in the generating the authentication tag, the authentication tag is a block cipher-based message authentication code.

10. The computer-implemented method according to claim 1, further comprising generating a final certificate including:

receiving challenge data from the second computing device;

receiving, by the key ladder, the encrypted set of device-specific data provided by a preceding generation process;

deriving, by the key ladder, a key from the UDS using said encrypted set of device-specific data as input data;

generating a final tag from the challenge data using the key; and

generating the final certificate including the challenge data and the final tag.

11. The computer-implemented method according to claim 1, wherein the deriving further includes executing a cryptographic algorithm including at least one of a decryption algorithm and an encryption algorithm, at each of a plurality of levels of the key ladder.

12. A computer implemented method for attesting the authenticity of a first computing device provisioned with a unique device secret “UDS”, by a second computing device provisioned with a key that is derived from the UDS of the first computing device, the method comprising:

receiving, by the second computing device, from the first computing device, an N chained certificate(s) generated by the method according to claim 1; and

verifying, by the second computing device, the validity of the N chained certificate(s),

wherein the verifying the validity of the N chained certificate(s) further includes a verification process, performed by the second computing device, that includes:

F) deriving, by a key derivation module, a key from said provisioned key;

G) generating a verification tag from a set of device-specific data included in a certificate to be verified using the key;

H) comparing the verification tag and an authentication tag included in the certificate to be verified;

I) if the verification tag and the authentication tag match, validating the certificate;

J) encrypting the set of device-specific data included in the certificate to be verified with the key; and

K) providing the encrypted set of device-specific data as an input to the key derivation module for a subsequent verification process.

13. The computer-implemented method according to claim 12, wherein said provisioned key corresponds to a higher-level key generated by the key ladder in the deriving the key.

14. The computer-implemented method according to claim 13, wherein, in a first verification process, the deriving, by the key derivation module, the key from the provisioned key includes receiving as input, by the key derivation module, a predetermined initial value that is the same as the predetermined initial value received by the key ladder as lower-level input in a first generation process.

15. A first computing device comprising:

at least one key ladder; and

circuitry configured to attest authenticity of the first computing device, said first computing device being provisioned with a unique device secret (“UDS”), by being configured to:

generate an N chained certificate(s), the N chained certificate(s) attesting the authenticity of an N set(s) of device-specific data of the first computing device, and transmit the N chained certificate(s) to a second computing device,

wherein the circuitry is further configured to generate the N chained certificate(s) by being configured to:

A) derive, using the key ladder, a key from the UDS.

B) generate an authentication tag from a set of device-specific data, using the key.

C) generate a certificate, said certificate including the set of device-specific data and the authentication tag.

D) encrypt the set of device-specific data with the key, and

E) provide the encrypted set of device-specific data as an input to the key ladder for a subsequent generation process.

16. A second computing device, comprising:

at least one key derivation module; and

circuitry configured to attest authenticity of the first computing device, said first computing device being provisioned with a unique device secret (“UDS”), by a second computing device provisioned with a key that is derived from the UDS of the first computing device, by being configured to:

receive, from the first computing device, an N chained certificate(s) generated by the first computing device according to claim 15, and

verify, by the second computing device, the validity of the N chained certificate(s), wherein the circuitry is further configured to verify the validity of the N chained certificate(s) by the circuitry being configured to:

F) derive, by a key derivation module, a key from said provisioned key,

G) generate a verification tag from a set of device-specific data included in a certificate to be verified using the key,

H) compare the verification tag and an authentication tag included in the certificate to be verified.

I) if the verification tag and the authentication tag match, validate the certificate,

J) encrypt the set of device-specific data included in the certificate to be verified with the key, and

K) provide the encrypted set of device-specific data as an input to the key derivation module for a subsequent verification process.

17. A system comprising the first computing device and the second computing device according to claim 16.

18. The system according to claim **17**, wherein the key ladder has at least two levels and the circuitry is further configured to derive the key from the UDS, using the key ladder, by being configured to: receive a higher-level input value, and derive a higher-level key from the UDS used as a key or from a UDS-derived key with the higher-level input value; and wherein the higher-level input value received by the key ladder includes a predetermined constant that is the same for all the generation processes.

19. A non-transitory computer readable medium having stored thereon a program that when executed by a computer causes the computer to implement the method according to claim **1**.

20. A non-transitory computer readable medium having stored thereon a program that when executed by a computer causes the computer to implement the method according to claim **12**.

* * * * *